

队伍编号	MCB2400673
赛道	B

基于多模型优化调参和 MOPSO 算法的商品库存管理

摘 要

在现代电商物流管理中，高效的库存管理和准确的销量预测对于维持运营效率和成本优势至关重要。本文通过建立预测模型，对350个品类进行了未来三个月的库存和销量预测，并根据预测结果通过构建0-1整数线性规划模型优化仓库存储策略。

针对问题一，首先根据销量数据的统计特征进行分类，接着对于数据缺失类商品分别使用**线性插值法**和**移动中位数法**进行数据缺失处理。在对于库存量的预测中，本文通过构建**随机森林模型**并通过**随机搜索法**进行模型性能优化，经过对模型**均方根误差检验**，验证调参后的模型具备更优精度，优化后模型预测的库存量结果见表1。在对于销量的预测中，首先通过**特征工程**构建特征，接着对于分类后的四类商品的数据特征，选择与其适配的多个模型进行结果预测:1.对于季节性波动类数据，通过五类时序模型比对，通过**网格搜索算法**对**Prophet模型**进行优化，最后通过**MAPE值检验**，**残差图图像分析**进行模型优化验证。2.对于单日波动剧烈型类数据，采用五类机器学习模型进行结果预测，基于**5折交叉验证法**评估模型性能，选择性能最优的**XGboost模型**进行求解。3.对于时间段前后差异较大类数据，通过构建**季节性特征**和**加权销量特征**提高Prophet模型预测准确度，最后对模型关键性能指标进行评估以确定结果的准确性。4.对于数据缺失类数据，采用**分位数回归模型**进行结果预测，通过**收敛性检验**验证结果的准确性。商品销量的预测结果见表6。

针对问题二，为优化仓库的品类分配，本文建立了一个**多目标0-1整数线性规划模型**，模型包括三个**约束条件**:1.一个仓库中只能有一个品类；2.所有品类的存放总量不得超过仓库的容量上限；3.每日销量不得超过仓库产能上限。模型的**目标函数**旨在最小化仓库租金成本、最大化产品关联度，以及最大化仓容和产能利用率。在模型求解过程中，采用**多目标粒子群优化算法**求解最佳的“一品一仓”分仓方案。经过**数理分析**，该方案所使用仓库的产能利用率平均超过**85%**，其中部分仓库达到**90%**以上，总租金成本为**387629.61元**。

针对问题三，本研究建立了一个**线性规划模型**以优化品类在多个仓库中的分布策略，模型包括以下四个**约束条件**:1. 每个品类的存放比例之和必须等于1；2. 每个品类最多可以存放在三个仓库内；3. 一个仓库内存放的所有品类的库存量不应该超过该仓库的容量上限；4. 仓库中所有品类的出库量不应超过其产能上限。该模型的**目标函数**旨在最大化仓库资源的利用效率和品类间的关联度，最小化运营成本，并尽量将相同高级品类和件型存放在同一仓库内。在模型求解过程中，采用**多目标粒子群优化算法**，并引入了**惩罚机制**以优化同一高级品类和件型的集中存放，提高总关联度。最终求解出最佳的“一品多仓”分仓方案，经过数理分析，该方案所使用的仓库中的高级品类最大值为**5**，件型最大值为**2**，满足题目对于同伴型同品类储存需要。

本文的亮点在于:1.问题一中对于销量数据进行细致分类，按照类别数据特点，通过多模型参数对比，选择最优模型进行算法优化，预测结果更为准确2.问题二、三中考虑仓容利用率、产能利用率指标等多项指标，经过分析验证，模型具有合理性和一定的现实意义。

关键词:Prophet 网格搜索 多目标0-1整数线性规划模型 多目标粒子群优化算法

目录

一、 问题重述	4
1.1 问题背景	4
1.2 问题提出	4
二、 问题分析	4
2.1 问题一的分析	4
2.2 问题二的分析	5
2.3 问题三的分析	5
三、 模型假设	5
四、 符号说明	5
五、 数据处理与分类	6
5.1 数据特征分类	6
5.2 数据处理	7
5.2.1 短期数据缺失的处理	7
5.2.2 边缘数据缺失的处理	7
六、 问题一模型的建立与求解	8
6.1 月库存量预测的模型建立与求解	8
6.1.1 随机森林模型的建立	8
6.1.2 随机森林模型的优化	9
6.1.3 模型的求解	10
6.2 日销量预测的模型建立与求解	11
6.2.1 特征构造	11
6.2.2 季节性波动类数据模型的建立	12
6.2.3 季节性波动类数据模型的结果比对	13
6.2.4 Prophet 分解的时间序列模型的算法优化	14
6.2.5 单日波动剧烈型类数据模型的建立	16
6.2.6 单日波动剧烈型类数据模型的结果分析	18
6.2.7 时间段前后差异较大类数据模型的建立与求解	22
6.2.8 数据缺失类数据模型的建立	23
6.2.9 数据缺失类数据模型的求解	23
6.2.10 七到九月销量预测结果展示	24
七、 问题二模型的建立与求解	25
7.1 问题二模型的建立	25
7.1.1 决策变量	25
7.1.2 目标函数	25
7.1.3 约束条件	26
7.1.4 模型总结	27
7.2 问题二模型的求解	27
7.2.1 多目标粒子群优化算法	27
7.2.2 最终解的选择	29
7.3 结果展示	30
7.4 结果可行性分析	30
八、 问题三模型的建立与求解	31

8.1 问题三模型的建立	31
8.1.1 决策变量	31
8.1.2 目标函数	32
8.1.3 约束条件	32
8.1.4 模型总结	33
8.2 问题三模型求解	34
8.3 问题三结果展示	35
8.4 问题三结果可行性分析	35
九、 模型的评价、改进与推广	37
9.1 模型的优点	37
9.2 模型的缺点	37
9.3 模型的推广	37
十、 参考文献	37

一、问题重述

1.1 问题背景

在电商物流体系中，商品的存储和管理依托于多个仓库，商品按照品类、件型等属性被合理划分，以便进行库存管理和物流调度。通过科学的货量预测和合理的品类分仓规划，电商企业可以有效降低库存成本，并提升仓储管理和订单履约的效率。

在库存管理中，需求预测是供应链决策的重要基础。通常需要根据历史数据对未来的库存量和销量进行预测，以便提前规划仓储资源，减少不必要的成本投入。然而，受多种因素影响，如仓库调整、促销活动等，需求量预测的准确性面临较大挑战，需要通过建立智能预测模型来尽可能提高预测的精准度。

合理的分仓规划还需要综合考虑仓库的存储容量和出库能力等约束条件，以避免仓库容量和产能的超负荷，确保各品类商品的合理分布，从而达到降低仓储和履约成本的目的。

1.2 问题提出

在此场景中，电商企业需要对其仓储网络中的 350 个品类进行货量预测和分仓规划，以提升整体运营效率。具体而言，需要解决以下三个问题：

（1）通过建立货量预测模型，预测未来三个月内（7 月至 9 月）各品类的月均库存量和每日销量，作为后续分仓规划的依据。

（2）在每个品类只能存放于一个仓库的前提下，制定最优的品类分仓方案，综合考虑仓库容量、产能和成本等因素，确定每个品类的存放位置。

（3）允许一个品类存放于多个仓库的情况下，制定新的分仓规划方案。此时，同一品类最多可以分配到三个仓库，并优先考虑相同件型和高级品类的集中存放，以最大化品类关联度，同时兼顾仓库容量、产能和仓储成本等业务指标。

二、问题分析

2.1 问题一的分析

针对问题一，我们的目标是对仓储网络中 350 个品类的库存量及销量进行预测。对于库存量，我们分析发现其数据完整且无大面积缺失，与随机森林模型的处理较为匹配，因而采用随机森林模型进行预测。接着，针对该模型，采用随机搜索的方法来调整模型的关键参数，进而实现模型的优化。经过模型均方根误差检验，验证调参后的模型具备良好的精准度，因而结果具有高度可行性。

对于日销量预测，首先，对日销量数据进行特征构造，提高预测准确度。接着，根据数据分类后的各类型数据特征，选择与其适配的多个模型进行结果预测，通过模型参数对比，选择最为合适的模型进行算法优化与调参。最后对优化后的模型进行关键参数验证，确保其结果具有准确性。

2.2 问题二的分析

问题二要求在一品一仓的前提下，考虑多个业务目标，基于问题一的预测结果建立分仓规划模型。对于问题二，首先要明确业务目标和约束条件，结合题目要求和实际生活，本文将最大化仓库品类关联度和最小化租金成本为主要目标，最大化仓库产能利用率和仓容利用率为次要目标，约束条件为一品一仓、仓库容量限制和产能限制。接着，通过定义决策变量、设计目标函数和约束条件建立 0-1 整数线性规划模，并通过多目标粒子群优化算法进行求解，得到分仓方案。最后，对得到的分仓结果进行统计分析，来检验模型规划结果的实用性和可靠性。

2.3 问题三的分析

问题三要求在保持品类关联度最大化的同时，还需尽量将同伴型、同高级品类的商品放在同一个仓库中，并且同一品类存放的仓库数量不能超过 3 个。问题三是在问题二的基础上，增加了新的目标和替换了约束条件。本文首先对约束条件进行了调整，将一品一仓约束条件改为一品多仓，但不可以超过三个仓库。接着，添加了惩罚机制，依据仓库内不同高级品类和件型个数，使得个数越多的仓库目标函数值下降，从而满足分仓要求。随后，在模型求解的过程中，本文修改了 MOPSO 算法对于群体极值的选取方式，使得品类关联度更大的分仓方案更易被选取，最后对得到的分仓方案进行数理分析，来验证结果的可靠性和合理性。

三、模型假设

1. 假设已给商品的库存量和销量准确度高，仅个别数据有误
2. 假设销量会受到节假日、商家促销等因素影响，导致一定的不规律性
3. 假设未来 3 个月的数据趋势与历史一致或相似

四、符号说明

符号	说明	单位
μ	均值	/
S	方差	/
$lag(i, j, n)$	品类 <i>i</i> 第 <i>j</i> 天的滞后 <i>n</i> 天的特征	/
$roll(i, j, n)$	品类 <i>i</i> 第 <i>j</i> 天的滚动 <i>n</i> 天的特征	/
MSE	均方误差	/
MAE	平均绝对误差	/
$RMSE$	均方根误差	/
R^2	决定系数	/
$<$	支配	/
x_{ij}	矩阵第 <i>i</i> 行第 <i>j</i> 列的元素	/
I_{im}	第 <i>i</i> 品类的第 <i>m</i> 月库存量	/
O_{in}	第 <i>i</i> 品类的第 <i>n</i> 天销量	/
E_i	第 <i>i</i> 号仓库的日租金	元
C_i	第 <i>i</i> 号仓库的容量	/
P_i	第 <i>i</i> 号仓库的产能上限	/
$R(a, b)$	品类 <i>a</i> 与品类 <i>b</i> 的关联度	/

五、数据处理与分类

5.1 数据特征分类

为了提高销量预测的准确性，我们对销量数据进行了系统性的分类处理。通过对数据的详细观察和分析，我们发现库存量数据较为完整，不存在缺失值和异常值；但对于不同品类的销量数据，存在显著的差异性和复杂性。为了有效应对这些差异性，我们将销量数据分为四类：数据缺失类、时间段前后差异较大类（特征工程）、单日波动剧烈型类（机器学习）、季节性波动类（基本解决）。以便在后续的预测过程中能够针对每一类别的特性选择合适的模型，从而提升预测的准确性和可靠性，分类算法步骤如下：

Step1:数据拆分

首先，将包含 350 个品类的销量数据文件附件 2.csv 分拆为单独的 csv 文件，每个文件代表一个品类。

Step2:初步筛选出销量数据数据缺失类

检查每个文件的体积，将小于 3KB 的文件标记为“数据缺失类”，表明文件中大量数据缺失。

Step3:时间序列不一致性检测

对于数据完整的品类，进一步分析 7-9 月与 4-6 月的销量数据。我们计算两个时间段的均值（ μ ）和方差（ S ），应用以下公式进行计算：

$$\left\{ \begin{array}{l} \mu_1 = \frac{1}{n_1} \sum_{i=1}^{n_1} x_i \\ \mu_2 = \frac{1}{n_2} \sum_{i=1}^{n_2} y_i \\ S_1 = \frac{1}{n_1 - 1} \sum_{i=1}^{n_1} (x_i - \mu_1)^2 \\ S_2 = \frac{1}{n_2 - 1} \sum_{i=1}^{n_2} (y_i - \mu_2)^2 \end{array} \right. \quad (1)$$

式中， x_i 和 y_i 分别为 7-9 月和 4-6 月的销量数据点， n_1 和 n_2 为两个时间段的数据点数。

如果满足以下条件之一，则将品类归为“两段时间销量分布不一致的品类”：

$$\begin{aligned} |\mu_1 - \mu_2| &> 2 \\ \frac{S_1}{S_2} &< \frac{1}{2} \text{ or } \frac{S_1}{S_2} > 2 \end{aligned} \quad (2)$$

Step4:单日波动性检测

对于未被前述步骤筛选的品类，计算整体时间段内的销量极差（ P ），以及所有时间段的平均销量（ μ ）：

$$\begin{aligned} P &= \max_{0 < i \leq n_1, 0 < j \leq n_2} \{x_i, y_j\} - \min_{0 < i \leq n_1, 0 < j \leq n_2} \{x_i, y_j\} \\ \mu &= \frac{n_1 \cdot \mu_1 + n_2 \cdot \mu_2}{n_1 + n_2} \end{aligned} \quad (3)$$

若 $P > 1.5\mu$ ，则归类为“单日波动剧烈型类”。

Step5:季节性趋势类判别

经过以上步骤后剩余的品类被认为是“季节性趋势类”。
通过细致分类方法，我们能够针对各类别数据特征选择最适宜的预测模型。此方法在提高预测准确性的同时，也使得我们能够更有效地应对不同数据特性，从而达到优化模型性能和提升预测效果的目的。

5.2 数据处理

在数据特征分类中，有数据缺失类数据，针对这一特点的数据，我们采取了两种策略来填补缺失值，确保数据完整性的同时维持其统计特征和波动性。具体策略如下：

5.2.1 短期数据缺失的处理

对于连续缺失三天以内的数据，我们选择线性插值法进行填补。该方法基于两个已知数据点之间的线性关系推断缺失值，假设缺失数据的两端已知点为 (x_1, y_1) ， (x_2, y_2) ，插值公式定义为：

$$y = y_1 + \frac{(y_2 - y_1)}{(x_2 - x_1)}(x - x_1) \tag{4}$$

式中 $x_1 < x < x_2$ ， $y_1 < y < y_2$ 。

5.2.2 边缘数据缺失的处理

对于长时间序列的缺失，特别是数据起始或结束阶段的缺失，我们采用移动中位数法进行处理。这种方法不仅可以填补较大的数据缺口，还能在一定程度上反映数据的波动特性，避免使用均值可能引入的平滑效应。

例如，若某品类在 2022 年 7 月初缺失 12 天的数据，我们首先对 7 月 13 日之后的数据应用线性插值法填补短期缺失，然后确定一个以 6 天为窗口大小（即缺失天数的一半）的滑动窗口。对于每一个缺失日，我们取其 6 天中的中位数作为该日的估计销量值。

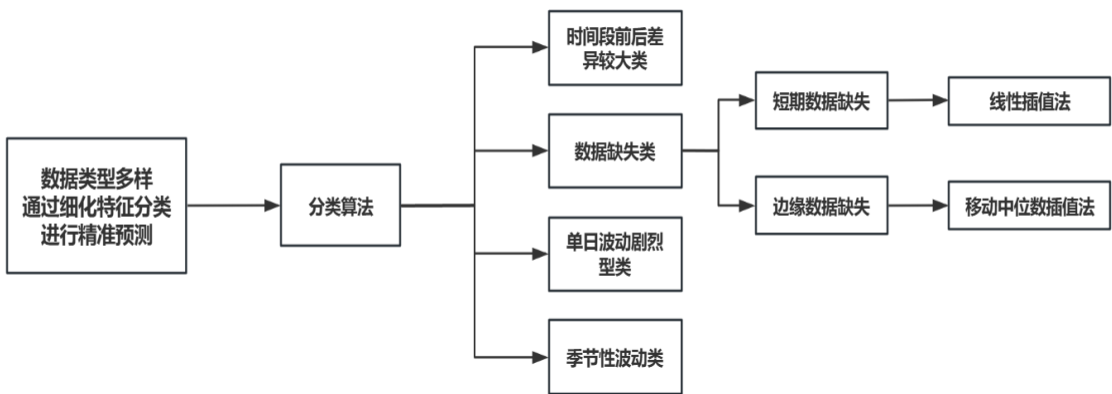


图 1 数据处理与分类部分流程图

六、问题一模型的建立与求解

6.1 月库存量预测的模型建立与求解

6.1.1 随机森林模型的建立

在对库存数据进行详细分析后，我们发现库存数据集涵盖了 2022 年 7 月到 9 月及 2023 年 1 月到 6 月的库存量，数据完整且无大面积缺失，这种数据的特性与随机森林模型的处理能力高度匹配。随机森林模型能够有效处理大量的输入特征，并且对异常值和非线性关系具有较强的鲁棒性。

随机森林是一种基于 Bagging (Bootstrap Aggregating) 策略的集成学习模型，它能够有效地处理非线性问题，并且擅长处理大量样本和特征。此外，随机森林还注重降低方差。Bagging 方法在训练过程中，各基学习器之间无依赖，可实现并行训练。通过集成多个模型，该模型可以有效地处理过拟合问题，提高模型的预测准确性和泛化能力，可应用于分类、回归和异常检测任务^[4]。随机森林模型的处理步骤如下：

为了充分利用随机森林模型处理库存量预测，我们按照以下步骤操作，并考虑了数据集的特定特征：

Step 1: 选择训练集

对于每棵树，从所有 N 个数据中随机地有放回地抽取 n 个样本作为训练集，允许一些样本在同一树的训练集中被多次抽取，从而保证了训练数据的多样性并减少模型的方差。

Step 2: 选择特征和计算分割标准

随机森林在构建每棵树的每个决策节点时，随机选择 m 个特征，并使用这些特征来决定最佳分割点。本文采用信息增益分割标准，帮助模型在每个节点处选择最能减少不确定性的特征，从而优化决策树的构建过程，具体公式如下：

$$IG(D, f) = Entropy(D) - \sum_{j=1}^k \frac{|D_j|}{|D|} Entropy(D_j) \quad (5)$$

Step 3: 树的生长

每棵树在没有任何剪枝的情况下尽可能完全生长，这样可以捕捉尽可能多的数据特征。这一步是为了确保每棵树能够从其训练集中学习到足够的信息，增强模型的预测能力，决策树生长过程如图 2 所示。

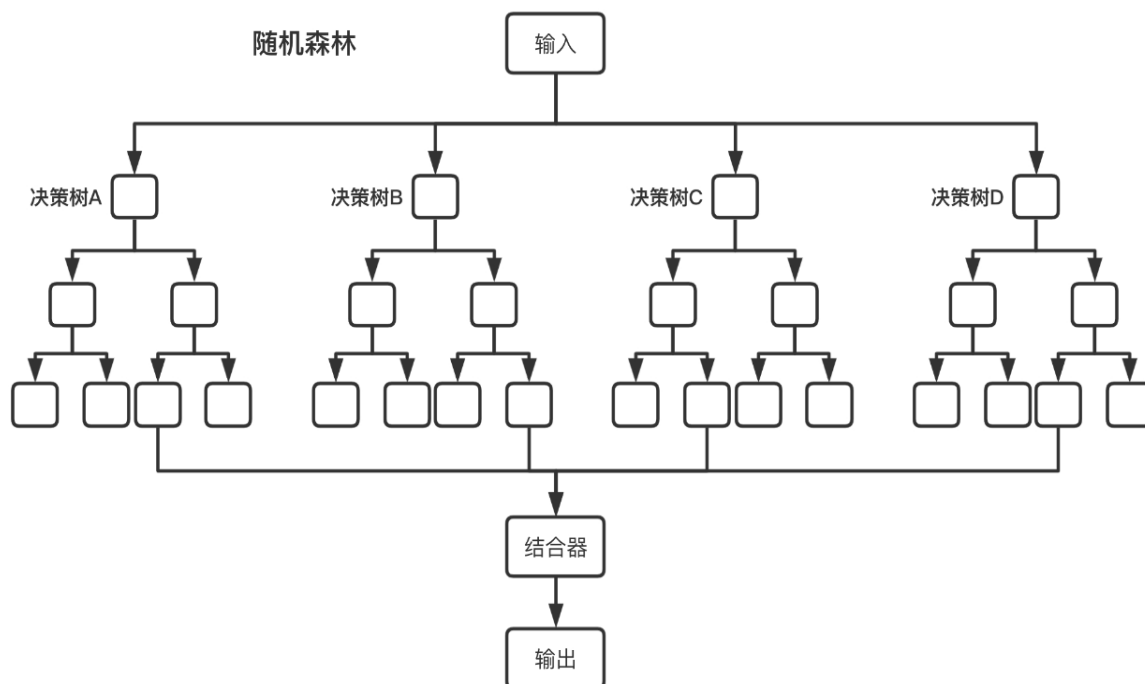


图 2 决策树生长示意图

Step 4: 聚合预测

最终的预测结果是通过聚合所有树的输出得到的。对于回归问题，结果通常是所有树预测结果的平均值；对于分类问题，则通过多数投票法确定最终结果。

6.1.2 随机森林模型的优化

为了进一步优化随机森林模型的性能，我们采用了随机搜索的方法来调整模型的关键参数。随机搜索是一种有效的参数优化策略，它通过在预设的参数范围内随机选择参数值，来找到最佳的模型配置。

Step 1: 定义参数空间:

- 树的最大深度: 控制每棵树的最大深度，影响模型的复杂度和拟合程度。
- 叶子节点的最小样本数: 决定了每个叶节点最少包含的样本数，可以防止模型在训练数据上过度拟合。
- 树的数量: 随机森林中树的数量，更多的树可以增加模型的稳定性和准确性。

Step 2: 执行随机搜索

使用参数设置，进行随机搜索，每次迭代随机选择一组参数，使用交叉验证来评估这些参数设置下模型的性能。这样不仅可以评估单个参数的影响，还能观察到参数之间的相互作用如何影响模型的最终表现。

Step 3: 选择最优参数

在多次迭代后，通过比较不同参数组合下的模型性能，选择表现最佳的参数设置。如图 3 所示，随着迭代次数增加，模型的均方根误差（RMSE）逐渐降低，并在一定数量后趋于稳定。这种趋势表明模型在增加决策树的数量时，预测误差减少，模型的稳定性和准确性得到了提升。

通过图 3，我们可以得出结论，调参后的随机森林模型，特别是在树的数量上的优化，显著提高了模型对库存量预测的准确性，因此预测结果具有较高准确性。

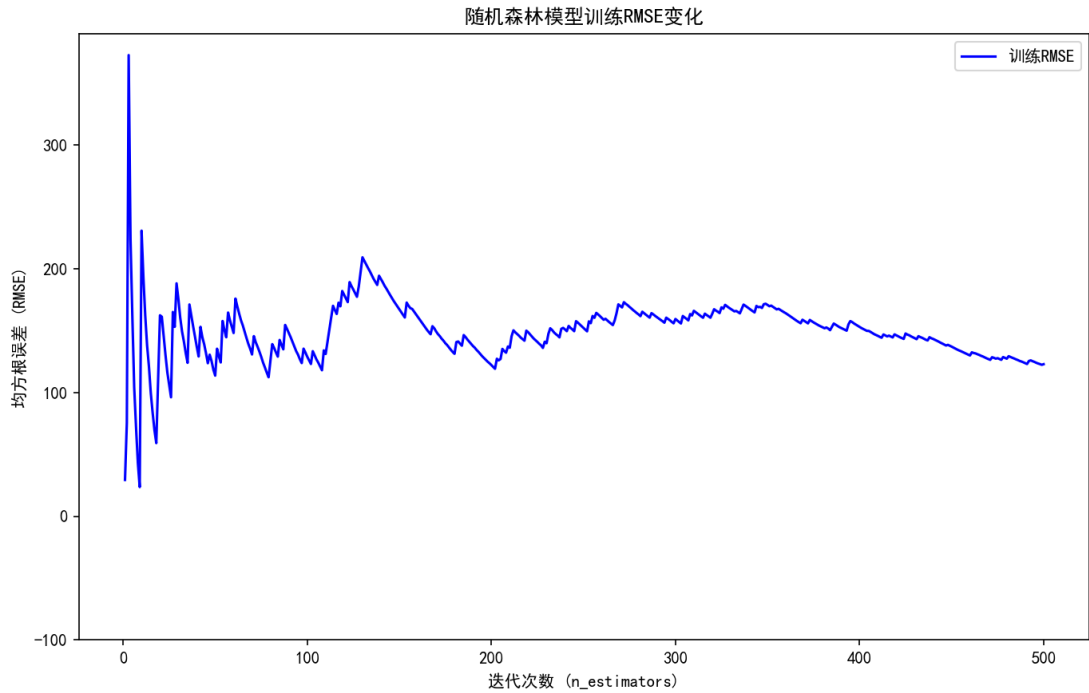


图 3 模型迭代系数与均方根误差关系

6. 1. 3 模型的求解

通过调参后的随机森林模型，对该仓储网络 350 个品类未来 3 个月（7-9 月）每个月的库存量进行预测，结果如表 1 所示

表 1:月库存量预测结果

	7 月库存量	8 月库存量	9 月库存量
category1	6916	6263	5705
category31	229	223	171
category61	247067	226270	209475
category91	14833	14670	14303
category121	87320	67684	78192
category151	131712	125365	133536
category181	241	273	354
category211	12882	10181	8758
category241	53716	43075	33690
category271	15021	14278	12875
category301	2488	2190	3139
category331	43	608	811

6.2 日销量预测的模型建立与求解

6.2.1 特征构造

通过有效的特征构造，我们不仅能够提取出与预测结果高度相关的信息，还能改善模型对复杂趋势的捕捉能力，从而提高预测的准确性。在本文中，我们通过构造以下四类新特征，以增强模型对季节性波动数据的适应性和预测能力。

(1) 时间特征:我们从原始日期特征中提取了五个关键的时间特征，包括年份、月份、周数、季度和星期几。这些时间特征为模型提供了捕捉季节性和周期性变化的基础。具体而言，不同的月份可能对应于不同的销售模式，而每周的销量往往受工作日和周末的影响。因此，这些时间特征能够帮助模型识别和利用销量变化中的周期性规律。

(2) 滞后特征:为了充分利用历史销量数据，我们构造了滞后特征，包括滞后一天和滞后两天的销量，令品类*i*第*j*天的滞后*n*天的特征为 $lag(i, j, n)$ ，品类*i*的第*j*天的销量为 O_{ij} ，其计算公式如下:

$$lag(i, j, n) = \sum_{k=1}^n O_{i(j-k)} \quad (6)$$

滞后特征能够反映当前销量与过去销量之间的关系，从而为模型提供有关短期趋势的重要信息。在时间序列分析中，滞后特征是理解数据动态变化的重要工具，有助于提升预测的准确性。

(3) 滚动特征:我们通过构造过去一周和过去一个月的滚动平均销量特征，平滑短期波动，使模型更加关注长期趋势，减少噪声和异常值对预测的影响。令品类*i*第*j*天的滚动*n*天的特征为 $roll(i, j, n)$ ，其计算公式为:

$$roll(i, j, n) = \sum_{k=1}^n O_{i(j-k)} / n \quad (7)$$

(4) 节假日特征:鉴于节假日对销量波动的显著影响，我们结合实际情况对日期范围内的重要节假日（如国庆节、五一劳动节和618促销节）进行了标记。这些节假日特征使模型能够捕捉因特殊事件导致的销量激增或下降情况，从而提高预测的准确性，具体标记日期如下:

表2 节假日标记

节假日	标记日期
中秋节	2022.9.9、2022.9.10、2022.9.11、2022.9.12
清明节	2023.4.4、2023.4.5、2023.4.6
五一劳动节	2023.5.1、2023.5.2、2023.5.3
6.18 促销节	2023.6.17、2023.6.18、2023.6.19
端午节	2023.6.21、2023.6.22、2023.6.23、2023.6.24

考虑到部分节日前就已经有购物潮这一因素，故将节日当天的前一天也纳入标记范围。

6.2.2 季节性波动类数据模型的建立

在提取季节性波动的数据后，我们根据其特征选择了五种适合时序分析的模型进行建模与求解，以期得到对该类数据最为适用的模型。

通过对比这五种模型在未调参情况下的预测准确度，我们旨在识别出表现最优的模型。在确定最优模型后，我们对该模型进行进一步的算法优化与调参，我们将进行进一步的算法优化与调参，以提升其预测性能和稳定性，最终获得最合理且准确的预测结果。本文通过大量的模型对比与选择性深入优化，确保了所选模型具有科学性和适用性，从而能够提供较为合理的预测结果。以下针对季节性波动数据建立的五种时序分析模型：

(1) ARIMA 时间序列模型的建立

ARIMA（自回归积分滑动平均）模型是一种广泛应用于时间序列预测的统计模型，特别适用于平稳性数据及其季节性调整。ARIMA 模型通过对历史数据进行自回归和滑动平均来捕捉数据的内在结构。该模型的核心在于以下数学公式：

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \dots + \phi_p Y_{t-p} + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \dots + \theta_q \varepsilon_{t-q} + \varepsilon_t \quad (8)$$

式中， Y_t 为当前时刻的观测值， ϕ 是自回归参数， θ 是滑动平均参数， ε_t 是白噪声。

在处理提取的季节性波动数据时，ARIMA 模型能够有效捕捉数据的趋势和季节性成分。通过对历史销量数据的自回归分析，ARIMA 模型帮助识别过去销量与当前销量之间的关系，并通过滑动平均平滑随机波动，以确保了模型能够对具有明显季节性波动的数据进行合理的预测。

(2) SARIMA 季节性时序模型的建立

SARIMA（季节性自回归积分滑动平均）模型是在 ARIMA 模型的基础上引入季节性因素的扩展，适用于具有明显季节性波动的时间序列数据。SARIMA 模型的数学表达式为：

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \dots + \phi_p Y_{t-p} + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \dots + \theta_q \varepsilon_{t-q} + \varepsilon_t + \Phi_1 Y_{t-s} + \Phi_2 Y_{t-2s} + \dots + \Phi_P Y_{t-Ps} + \Theta_1 \varepsilon_{t-s} + \Theta_2 \varepsilon_{t-2s} + \dots + \Theta_Q \varepsilon_{t-Qs} \quad (9)$$

式中， s 为季节周期， ϕ 和 θ 是季节性自回归和滑动平均参数。

针对提取的季节性波动数据，SARIMA 模型通过捕捉季节性成分的变化，使得模型能更好地反映季节对销量的影响。这使得 SARIMA 在处理诸如本题目中电商销量等具有周期性变化的数据时，能够提供更准确的预测。

(3) Prophet 分解的时间序列模型

Prophet 是由 Facebook 开发的一种时间序列预测工具，特别适合具有强季节性和假期效应的数据。其模型公式为：

$$y(t) = g(t) + s(t) + h(t) \quad (10)$$

式中， $g(t)$ 是趋势组件， $s(t)$ 是季节组件， $h(t)$ 是假期效应。

在分析季节性波动数据时，Prophet 模型通过分解时间序列为趋势、季节性和假期效应，能够灵活适应不同类型的季节性变化。这一特性使得 Prophet 在处理本题中季节性波动趋势的数据时具有明显的优势。

(4) 基于 KNN 的预测模型构建

K 近邻算法是一种常用的回归和分类方法。基本思想是通过寻找与给定样本最近的 K 个邻居来预测其值。其数学表达式为:

$$\hat{y} = \frac{1}{K} \sum_{i=1}^K y_i \tag{11}$$

式中， $\hat{y}$ 是预测值， y_i 是K个邻居的实际值。

在处理季节性波动数据时，KNN 模型可以通过对销量数据的特征进行相似度计算，识别与当前数据点相似的历史样本，从而进行预测。这种方法能有效捕捉季节性变化的局部特征，使得模型在本类型数据中较为适用。

(5) 基于 SVM 的预测模型构建

支持向量机（SVM）是一种强大的监督学习方法，适用于回归分析。SVM 通过在高维空间中寻找最优超平面来进行预测，其数学公式为:

$$f(x) = w^T \varphi(x) + b \tag{12}$$

式中， w 是权重向量， $\varphi(x)$ 是特征映射， b 是偏置项。

在季节性波动数据的预测中，SVM 能够通过构建适当的核函数，捕捉数据中的非线性关系。这使得 SVM 在处理复杂的季节性波动时，具备了较高的预测能力。

6.2.3 季节性波动类数据模型的结果比对

在分别采用五个模型对数据进行处理后，我们将根据均方误差（MSE）、平均绝对误差（MAE）、均方根误差（RMSE）和决定系数（R²）这四个评估指标来判断模型的预测效果，以下是评估指标的说明:

①均方误差（MSE）:均方误差是指参数估计值与参数真值之差平方的期望值。它能够衡量预测值与实际值之间的差异，MSE 的值越小，说明预测模型在描述实验数据时具有更好的精确度。

②平均绝对误差（MAE）:平均绝对误差是指预测值与实际值之间绝对误差的平均值。MAE 提供了一个直观的误差评估，表示了预测值偏离实际值的平均程度。

③均方根误差（RMSE）:均方根误差是均方误差的算术平方根，表示预测值与实际值之间差异的标准偏差。较小的 RMSE 值表明模型的预测结果更接近真实值，适合于评估预测模型的整体表现。

④决定系数（R²）:决定系数表示模型对数据变异的解释程度，其值介于 0 与 1 之间。R²越接近 1，说明模型能够解释的变异越多，拟合效果越好。五个模型的性能指标数值如表 3 所示。

表 3 模型性能指标

	ARIMA 模型	SARIMA 模型	Prophet 模型	KNN 模型	SVM 模型
均方误差	174664.88	636747.45	23050.32	44820.71	109333.07
平均绝对差	176.15	303.41	57.08	77.92	122.89
均方根误差	226.86	349.29	78.3	109.4	169.99

决定系数	-0.84	-2.44	0.77	0.56	0.21
------	-------	-------	------	------	------

通过对表 3 性能指标的分析，可以得出不同模型预测的准确性差异。首先，ARIMA 模型和 SARIMA 模型均表现出较高的均方误差和负的决定系数，尤其是 SARIMA 模型，其 MSE 达到 636747.45，显示出模型无法有效捕捉数据中的趋势与季节性，导致预测结果偏差显著。

KNN 模型和 SVM 模型的性能指标虽然优于 ARIMA 模型和 SARIMA 模型，但仍不及 Prophet。KNN 的 MSE 为 44820.71，MAE 和 RMSE 也高于 Prophet，显示出其在捕捉数据的季节性波动方面的不足。SVM 模型的表现则更为有限，其 MSE 为 109333.07，说明模型在处理此类数据时的适应性较差。

相比之下，Prophet 模型的各项指标均表现优异。其均方误差（23050.32）显著低于其他模型，表明其预测结果与实际值之间的差异最小。平均绝对误差（57.08）和均方根误差（78.30）同样反映了其较高的预测精度。此外，Prophet 的决定系数（0.77）接近 1，表明该模型能够很好地解释数据的变异，具备良好的拟合能力。

综上所述，Prophet 模型在处理本题目中具有季节性波动的数据时表现出色，其准确性和稳定性均优于其他模型。通过低均方误差、平均绝对误差和均方根误差，以及较高的决定系数，Prophet 模型显然是此次预测任务的最佳选择。在下一部分我们将对 Prophet 模型进行进一步的算法优化和调参，以获得更优的预测结果。

6.2.4 Prophet 分解的时间序列模型的算法优化

为了优化 Prophet 模型以提高对季节性波动数据的预测精确度，我采用了网格搜索方法来调整关键的超参数^[5]。网格搜索是一种广泛使用的参数优化技术，它系统地遍历多种参数组合，通过交叉验证来确定最佳的模型配置。

（1）参数调整：

- **seasonality_prior_scale**:控制季节性影响的强度。较大的值使模型对季节性变化更为敏感，适用于数据中明显的季节波动。
- **changepoint_prior_scale**:影响趋势变化点的灵活性。数值越大，模型对趋势变动的适应性越强，能更快地响应数据中的突变点。
- **holidays_prior_scale**:设置假期效应的强度。通过增加该参数的值，模型将更加重视假期对销量的潜在影响。
- **n_changepoints**:趋势变化点的数量，指定模型在整个时间序列中应识别的潜在变化点的数量，对模型捕捉趋势变化至关重要。
- **seasonality_mode**:决定季节性效应是加性的还是乘性的。加性模式用于趋势变化平稳的序列，而乘性模式适合趋势随季节强度变化的序列。

（2）优化过程

通过使用交叉验证来评估不同参数组合的效果，优化目标是最小化预测误差，同时提高模型在未见数据上的泛化能力。通过网格搜索详尽地探索参数空间，确保找到 Prophet 分解的时间序列模型在当前数据集上表现最佳的模型配置。

（3）调参前后模型对比

◆ MAPE 值比较

MAPE 是一个常用的预测准确性度量，表示预测值和实际值之间的差异占实际值比例的平均值，通常用于衡量预测模型的准确度。较低的 MAPE 值意味着预测结果与实际数据更为接近，因此，准确度更高。

通过对比调整参数前后 Prophet 模型的平均绝对百分比误差（MAPE）值，来验证模型性能的提升。如图 4，调整参数后的 Prophet 模型（蓝色线）与调整参数前的模型（橙色线）的 MAPE 值进行了比较。从图中可以得出，在大部分时间点上，调

整参数后的模型 MAPE 值较低，尤其是在数据波动较大的时期。这表明通过网格搜索优化的参数设置有效地提高了模型对季节性和非线性变化的捕捉能力，从而使模型预测的准确性得到了显著提升。

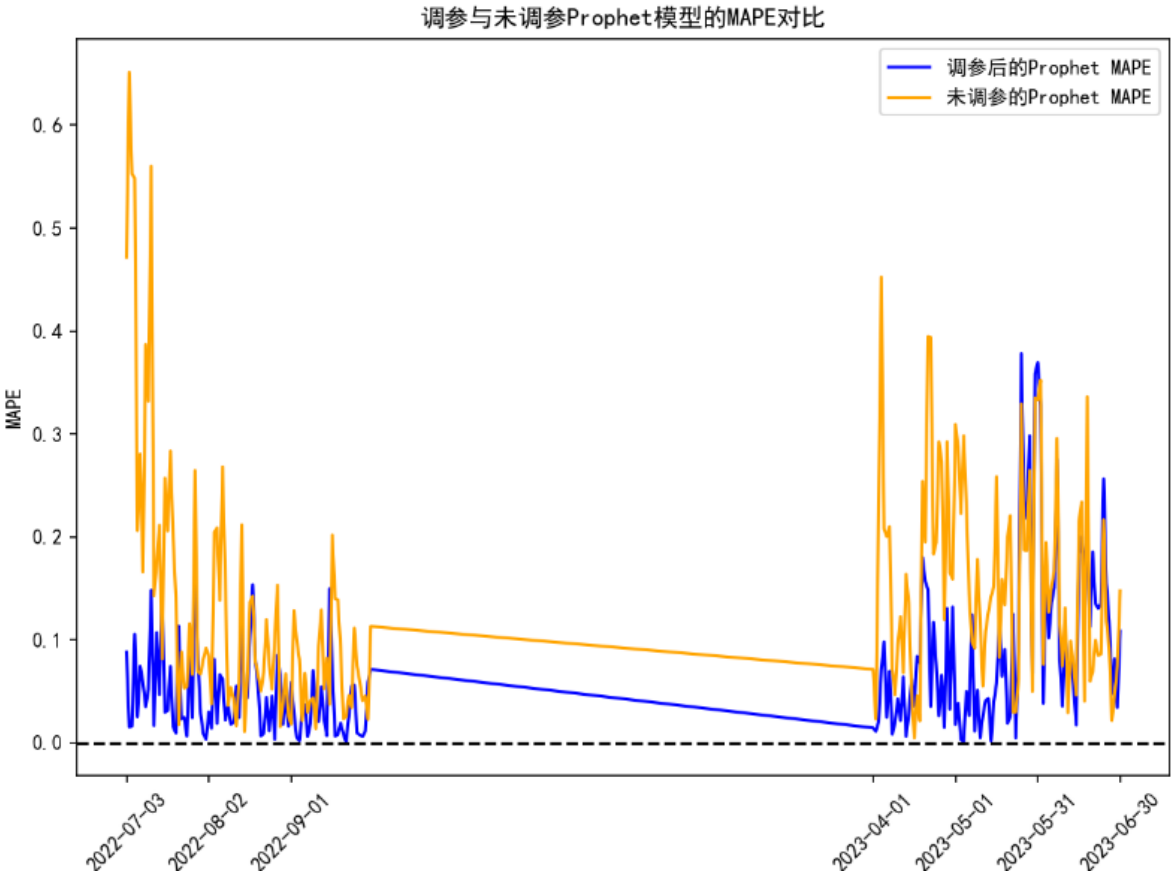


图 4 调参前后 MAPE 值比较

◆ 调参后模型的真实值与测试值对比图与残差图分析

①置信区间图分析:如图 5 真实值与测试值对比图所示，模型的预测值在大多数时间段内与真实值紧密对应，并且几乎全部位于 95%的置信区间内。这表明模型对数据的波动有很好的捕捉能力，调参后的模型预测精度高，且预测结果具有较高的可靠性。

②残差图分析:如图 6 残差图所示，残差在零附近随机分布，没有明显的系统性偏差或趋势。这种分布表明模型的预测误差小，且误差没有自相关性，表明模型对数据的拟合效果良好。

综上所述，调参后的 Prophet 模型在预测性能上有明显提升，显示出良好的稳定性和准确性，适合用于预测具有明显季节性变化的销量数据，在后续使用调参后的 Prophet 模型所得出的预测结果具有高度可行性。

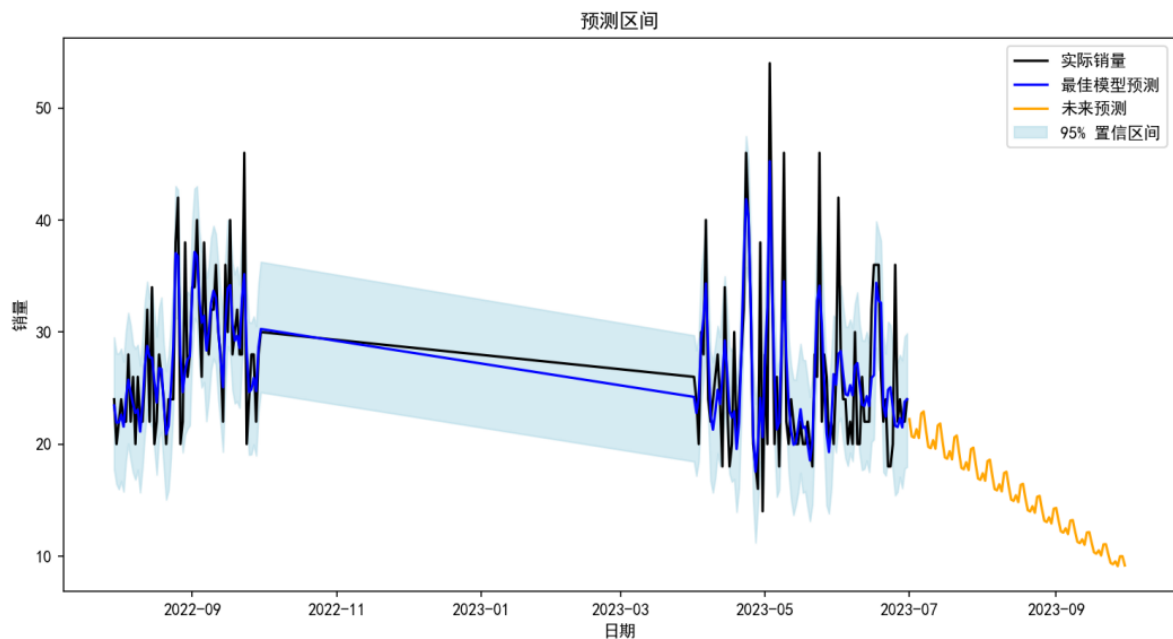


图 5 调参后模型真实值与测试值比较

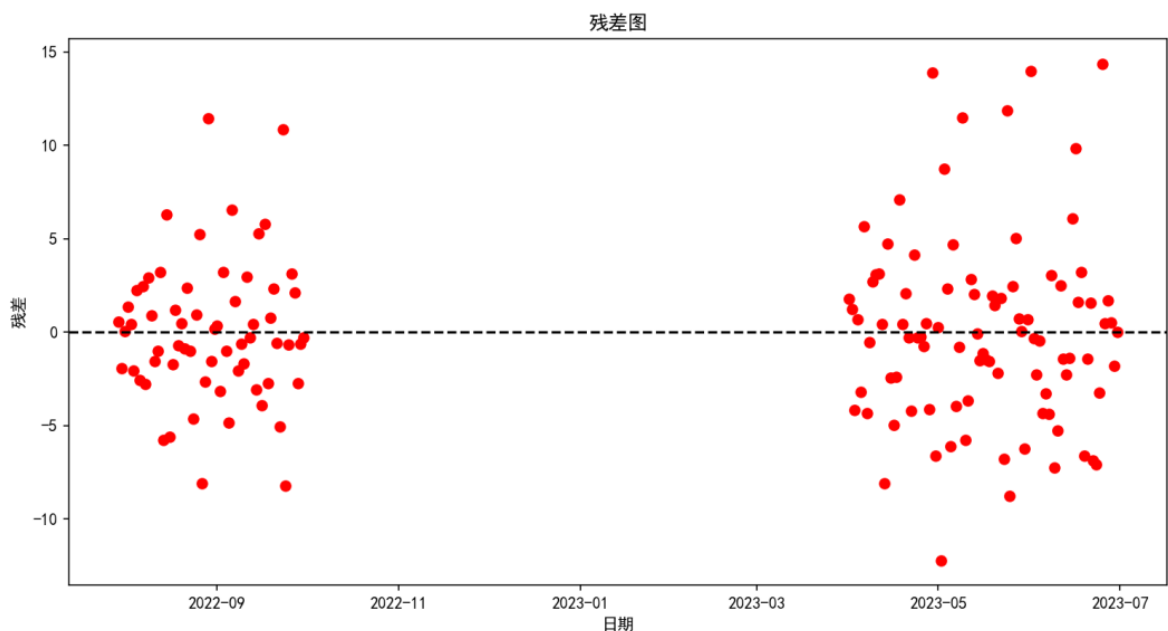


图 6 调参后模型残差图

6.2.5 单日波动剧烈型类数据模型的建立

在提取日趋势变化较大的数据后，我们选择了五种适合处理此类数据的机器学习模型进行建模与求解。机器学习算法适合于捕捉复杂的非线性关系和动态模式，能够有效应对日销量中的剧烈波动和高噪声特性。

通过对这五种机器学习模型的性能指标进行比较，我们将选择表现最佳的模型，进一步进行算法优化与调参，以提升预测的准确性和稳定性。以下是我们选用的五种机器学习模型：

(1) LightGBM 模型

LightGBM (Light Gradient Boosting Machine) 是一种高效的梯度提升决策树算法^[4], 广泛应用于大规模机器学习任务。其核心思想是通过构建多棵决策树, 并逐步修正前一棵树的预测误差, 以提高模型的整体性能。

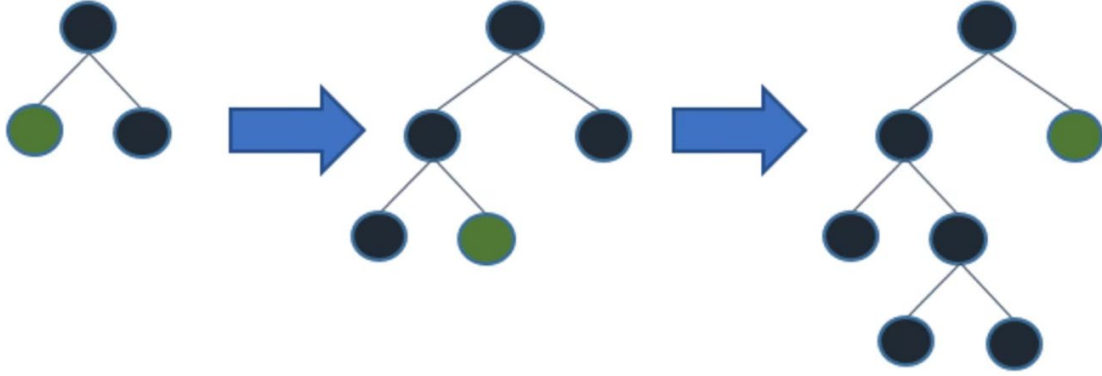


图 7 决策树生长

LightGBM 采用基于直方图的学习方法, 显著提高了训练速度和内存使用效率。目标函数可以表示为:

$$L(\theta) = \sum_{i=1}^N (y_i - F(x_i))^2 + \lambda \sum_{j=1}^M \theta_j^2 \quad (13)$$

其中, L 为损失函数, y_i 为真实值, $F(x_i)$ 为模型的预测值, λ 为正则化参数。LightGBM 适合处理日销量数据, 因为它能够迅速适应短期内的波动特征, 具备处理高维特征和大数据量的能力, 能够有效捕捉销量数据中的复杂模式和变化趋势。

(2) XGBoost 模型

XGBoost (Extreme Gradient Boosting) 是一个强大的集成学习算法, 主要用于分类和回归问题。通过构建多棵决策树^[1], XGBoost 能够有效地降低模型的偏差和方差, 从而提高预测的准确性。该模型通过正则化和并行化计算提升训练效率, 减少过拟合的风险。其目标函数可表示为:

$$L(\phi) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_k \Omega(f_k) \quad (14)$$

其中, L 为损失函数, $\hat{y}_i$ 为预测值, $\Omega(f_k)$ 反映了模型的复杂度。对于日销量预测, XGBoost 能够灵活捕捉因季节性、促销活动等外部因素引起的销量波动, 从而提供高精度的预测结果。

(3) 岭回归模型

岭回归是一种扩展线性回归的方法, 旨在解决多重共线性问题和防止过拟合。通过在损失函数中增加 L2 正则化项, 岭回归能够提高模型的稳定性和泛化能力。其损失函数的表达式为:

$$\hat{\beta} = \operatorname{argmin} \left(\sum_{i=1}^N (y_i - X_i \beta)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right) \quad (15)$$

在日销量数据预测中，岭回归可以有效利用历史销量数据，减少因异常波动引起的预测误差，从而增强模型的可靠性。尤其是在数据存在噪声时，岭回归的表现更为稳定。

(4) RNN 模型

递归神经网络（RNN）是一种专为处理序列数据而设计的神经网络，能够有效捕捉时间序列中的动态特征。通过在时间步之间传递隐藏状态，RNN 可以学习历史信息并将其用于当前的预测。RNN 的更新公式为：

$$\begin{cases} f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\ i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\ C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \\ o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\ h_t = o_t \cdot \tanh(C_t) \end{cases} \quad (16)$$

在处理日销量数据时，RNN 能够灵活适应快速变化的销售趋势，捕捉到数据中的短期波动和复杂模式。这使得 RNN 在需要分析时间序列的场景中表现出色，尤其适合于动态销售环境的建模。

(5) LSTM 模型

长短期记忆网络（LSTM）是 RNN 的一种改进，旨在解决传统 RNN 在处理长序列数据时的梯度消失问题。LSTM 通过引入记忆单元和门机制，有效捕捉长期依赖关系。其关键公式如下：

$$h_t = f(W_h h_{t-1} + W_x x_t + b) \quad (17)$$

在日销量波动的情况下，LSTM 能够灵活适应销量的变化，特别是在捕捉由于节假日、促销活动等因素引起的销量变化时，展现出优越的表现。通过学习长期和短期的依赖关系，LSTM 能够提供准确的销量预测，有助于制定有效的销售策略。

6.2.6 单日波动剧烈型类数据模型的结果分析

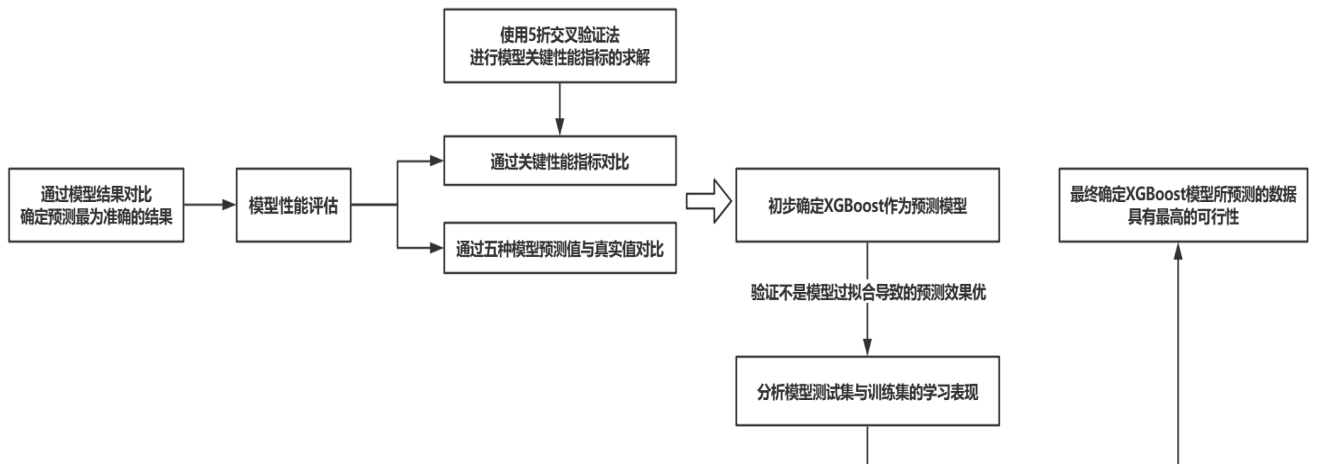


图 8 结果对比与模型选择流程

(1)K 折交叉验证

为了确保模型评估的可靠性，本文采用了 k 折交叉验证的方法。基于本文数据特征，选择进行 5 折交叉验证，更好地检测模型在不同数据划分下的表现，5 折交叉验证具体步骤如下：

Step 1:数据集划分

我们将原始数据集随机划分为五个大小相近的子集。这种划分方式确保每个子集在统计特性上尽量保持一致，从而避免因某个特定子集的偏差而影响模型的整体评估。

Step 2:交叉训练与验证

在每次迭代中，我们将选取其中的四个子集作为训练集，用于训练模型，而将剩余的一个子集作为验证集，来评估模型的性能。这个过程会重复五次，每个子集都将作为验证集一次。每次训练后，我们记录模型在验证集上的性能指标:均方误差 (MSE)、平均绝对误差 (MAE)、均方根误差 (RMSE) 和决定系数 (R²值)，从而进行更为精确的模型评估。



图 9 5 折交叉验证过程

(2) 模型性能评估

利用 5 折交叉验证，求解出来五个模型的关键性能指标如表 4 所示

表 4 模型性能评估

	LightGBM 模型	Ridge 模型	XGboost 模型	RNN 模型	LSTM 模型
均方误差	4327.056	2855.055	1456.788	2750.3261	2826.639
平均绝对差	42.643	29.439	23.517	33.945	33.874
均方根误差	65.008	51.073	37.400	51.327	51.983
决定系数	-0.276	0.236	0.577	0.216	0.198

通过对表 4 结果的分析，我们得出以下结论：

①均方误差 (MSE):XGBoost 模型表现最佳，MSE 为 1456.788，远低于 LightGBM 的 4327.056，RNN 的 2750.3261，和 LSTM 的 2826.639，表明其预测误差的平方和最小,显示了 XGBoost 模型在准确捕捉实际销量趋势方面的高效能。

②平均绝对误差 (MAE):XGBoost 同样表现优异, MAE 为 23.517, 这比 LightGBM 的 42.643、RNN 的 33.945 和 LSTM 的 33.874 都要低, 意味着 XGBoost 在预测销量时与实际销量的平均偏差最小。

③均方根误差 (RMSE):XGBoost 的 RMSE 为 37.400, 也是五个模型中最低的, 说明其预测结果的可靠和准确性。

④决定系数 (R^2 值):XGBoost 的 R^2 值为 0.577, 是唯一一个显著正值的模型, 表明 XGBoost 能够解释超过 57%的数据变异, 远超其他模型, 如 LightGBM 的-0.276, RNN 的 0.216 和 LSTM 的 0.198, 这表示 XGBoost 在数据拟合上的效果显著优于其他模型。

通过这些数据的综合比较, 我们可以明确地看到 XGBoost 模型在所有关键性能指标上的表现均优于其他模型, 表明 XGBoost 在处理具有剧烈日趋势变化的销量数据时, 不仅预测误差小, 而且拟合度高, 能够提供更准确和可靠的预测结果。

因此, 我们可以得出结论: XGBoost 模型在预测日趋势变化剧烈型销量数据方面表现最优, 但选择 XGBoost 模型仍需综合考量其与真实结果的对比。

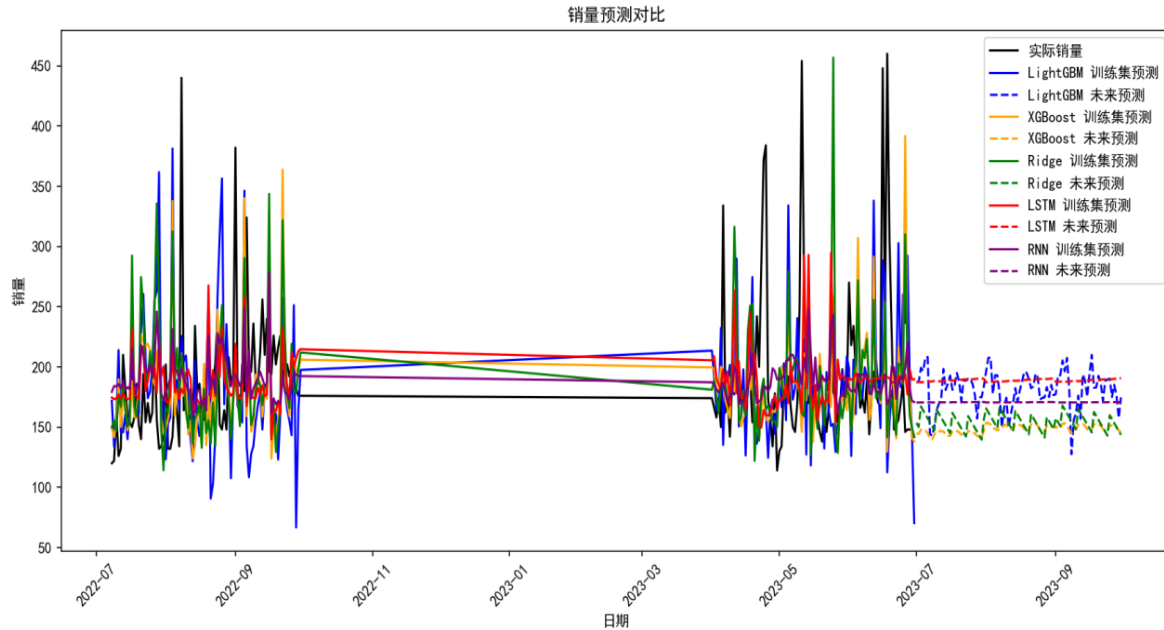


图 10 五种模型对 2022 年 7 月到 2023 年 9 月销量的预测结果与真实结果对比

根据图 10, 我们通过对比五种机器学习模型在 2022 年 7 月至 2023 年 9 月期间的预测结果与真实销量数据, 并将其预测结果与真实销量数据对比, 从而评估各模型的性能。经过分析我们发现 XGBoost 模型表现最优, 具体表现为: 五种模型与真实数据均展示了相同的趋势, 但在销量波动显著的时段, 如 2022 年末到 2023 年初, XGBoost 模型能够精准捕捉到销量的尖峰和低谷, 而其他模型如 LightGBM、岭回归、LSTM 和 RNN 虽显示出对总体趋势的追随, 却在峰值和谷底的捕捉上显得滞后或过度平滑。因此得出结论, XGBoost 较为适合处理具有波动性的时间序列数据, 所以初步选择 XGBoost 作为预测日趋势变化剧烈型销量数据的模型。

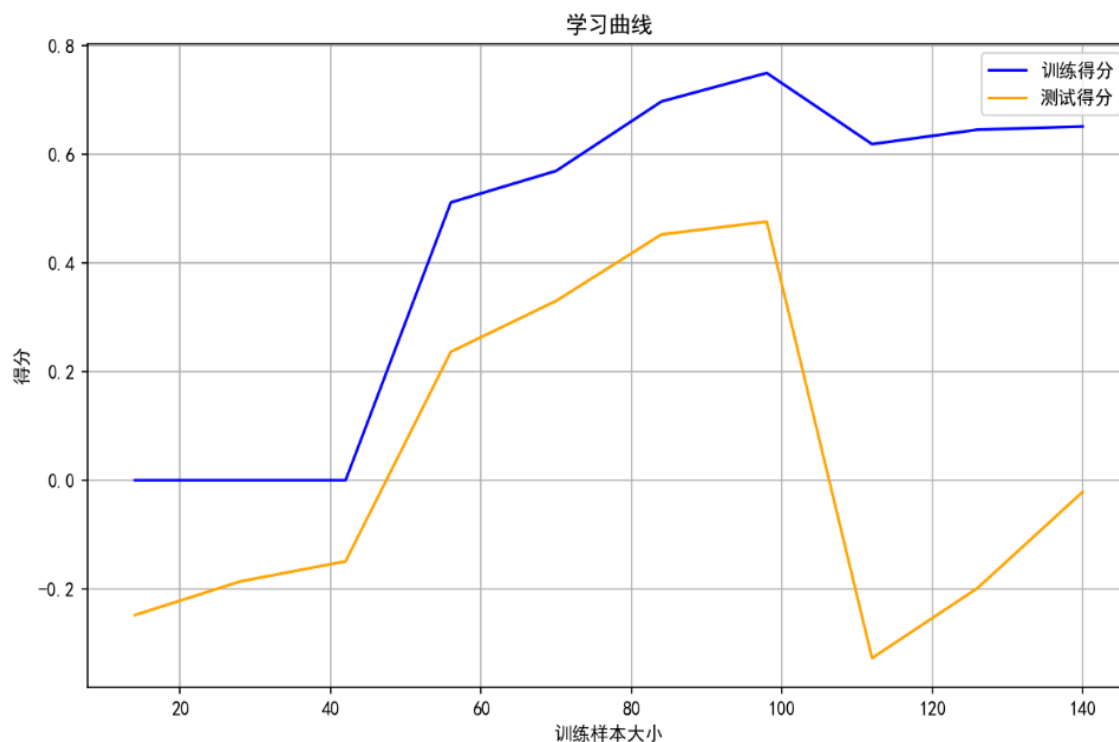


图 11 XGBoost 模型学习曲线

尽管 XGBoost 模型在五种机器学习模型中表现最佳，但为了模型的准确性与后续优化方向的正确，我们需要确保模型的良好效果不是由过拟合导致的，因此我们需要通过进一步分析图 5 中模型在训练集和测试集上的表现。

从图 11 中可以观察到，随着训练数据量的增加，训练得分和测试得分均有显著提高，尤其是在数据点超过 60 后，测试得分快速上升，表明模型对新数据具有很好的泛化能力。虽然训练得分始终高于测试得分，但两者的差距随着数据量的增加而逐渐缩小，这表明 XGBoost 模型并没有在训练数据上过度拟合，而是有效地学习了数据中的普遍规律。

在训练数据量达到 100 左右时，测试得分达到高峰后有所下降，我们猜测可能是由于训练样本中的某些噪声或异常值的影响。然而，测试得分很快稳定并保持在较高水平，这进一步验证了模型的鲁棒性和预测的可靠性。

综上所述，我们得出结论:XGBoost 模型在本研究中表现出的优异性能并非过拟合的结果。相反，XGBoost 模型具有良好的泛化能力能够有效地对日趋势变化剧烈型销量数据进行预测，因此最终选择 XGBoost 模型预测的结果。

6. 2. 7 时间段前后差异较大类数据模型的建立与求解

对于该类数据，我们选取较为典型的类别商品进行特征分析，如图 12 所示我们发现该类数据在 2022 年 7 月至 8 月和 2023 年 4 月至 6 月两个阶段的销量数据差距过大，因此我们推测此类商品的销量可能和季度或者月份等其他特征有关，因此通过特征构建以提高模型准确度。

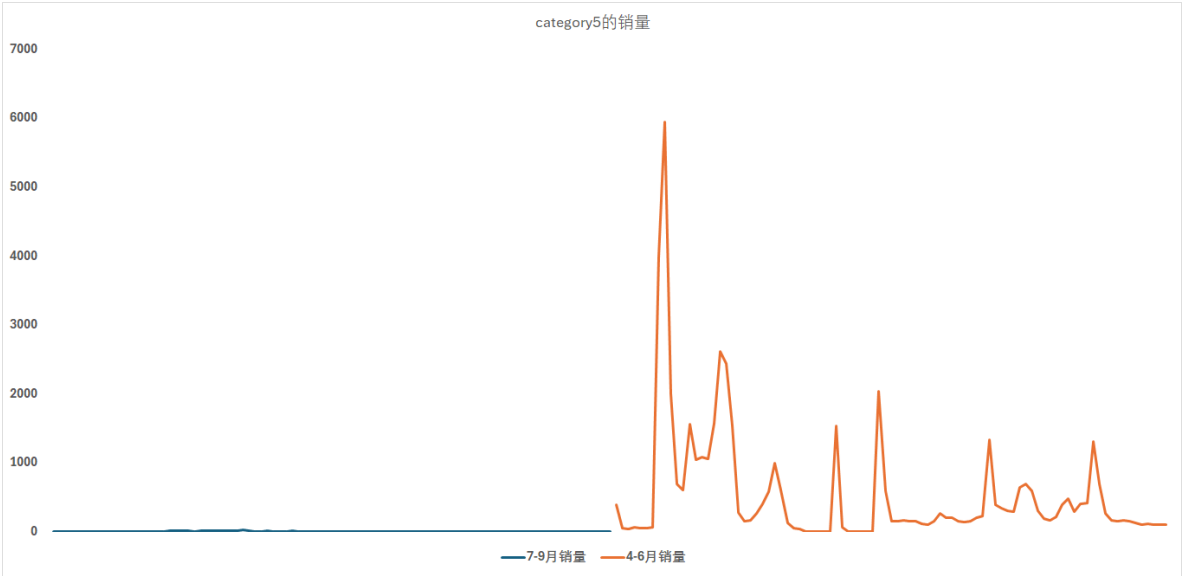


图 12 类别 5 的销量与时间关系

我们采取了以下策略进行特征构建:

● 季节性特征标记:

方法:引入一个新的二元特征来标记销量日期是否属于 7、8、9 月份，即第三季度。

目的:如果销量数据的日期属于第三季度，则该特征标记为 1，否则为 0。这有助于模型识别并适应销量的季节性波动，因为这一时间段可能因为特定的市场活动或消费者行为模式而销量增加。

● 加权销量特征:

定义:对第三季度的销量数据赋予更高的权重，即在模型训练过程中，将这些月份的销量权重提高到 1.5 倍。

目的:通过这种加权方法，模型可以更重视影响销量的关键季节性因素，从而在预测时能够更精确地反映出季节变化对销量的实际影响。

模型精度验证:

对经过特征构建的 preohet 模型的残差分析，计算出模型的残差均值、标准差、偏度和峰度，我们可以评估模型的性能和预测精度，具体数据如表 5 所示。

表 5 preohet 模型的残差数据

残差均值	0.30
残差标准差	5.08
残差偏度	0.12
残差峰度	3.89

以下是对表中统计指标的具体分析:

①**残差均值 (0.30)**: 残差均值接近于 0，说明模型的预测结果与实际值之间的偏差较小，表明模型整体上没有系统性偏差，且平均误差很小。

②**残差标准差 (5.08)**: 残差的标准差衡量了预测误差的波动性。虽然 5.08 的值表明预测结果与实际数据之间存在一定的波动，但这在复杂的预测模型中是合理的。

③**残差偏度 (0.12)**: 偏度衡量数据分布的对称性。0.12 的偏度值接近于 0，表明残差分布是相对对称的，说明模型没有系统地高估或低估预测值。

④**残差峰度 (3.89)**: 峰度是衡量数据分布尖峭程度的统计量。3.89 的峰度比正态分布的峰度（即 3）稍高，表明残差分布比正态分布稍微尖锐一些，这可能意味着数据中虽存在一些极端值或异常值，但并不严重。

综合这些统计指标，我们可以得出结论，经过特征优化的 preohet 模型具有较高的预测精度，其结果具有高度可行性。

6.2.8 数据缺失类数据模型的建立

对于数据缺失类，特别是那些基本为零或数据量过小的商品类别，我们选择分位数回归模型来处理这些数据。传统的回归模型通常依赖于大量数据来估计参数，因此可能导致使用传统回归模型时出现较大的拟合偏差。而分位数回归模型关注不同的数据分布区间，能够提供关于数据不同条件分位数的信息来优化模型的预测能力，因此我们选择分位数回归模型来预测数据缺失类数据的未来趋势。

(1) 分位数回归模型构建

分位数回归模型通过最小化分位数损失函数来适应数据的特定分位数，这一点与传统的最小二乘回归模型（聚焦于平均情况）不同。具体来说，分位数回归的目标是最小化以下形式的损失函数：

$$\min_{\beta} \sum_{i=1}^n (\tau - 1(y_i - x_i^T \beta < 0))(y_i - x_i^T \beta) \quad (18)$$

式中， τ 是我们关注的分位数， β 是模型参数， x_i 是解释变量， y_i 是响应变量。对于处理数据缺失或极端值影响较大的情况，模型能够更灵活地捕捉数据中的非典型行为。

6.2.9 数据缺失类数据模型的求解

Step 1: 模型参数初始化

为模型设定初始参数。这包括为回归系数 β 设定初始估计值，通常可以从简单的线性回归模型获取或设为零。

Step 2: 损失函数的计算与优化

利用分位数回归的损失函数进行模型求解。具体的损失函数为：

$$L(\beta) = \sum_{i=1}^n \rho_{\tau}(y_i - x_i^T \beta) \quad (19)$$

式中， τ 是选择的分位数， x_i 是观测值的解释变量， y_i 是相应的响应变量， β 是模型参数。

Step 3: 迭代求解

通过迭代方法（如梯度下降或更高级的优化算法）更新模型参数，以最小化损失函数。在每次迭代中，根据当前参数计算梯度，并更新参数值：

$$\beta_{new} = \beta_{old} - \alpha \nabla L(\beta_{old}) \quad (20)$$

式中， α 是学习率，决定了参数更新的步长。

Step 4: 收敛性检验

检查参数的更新是否满足收敛条件，检查参数更新的大小是否小于预设的阈值，或损失函数的改变是否足够小。

Step5: 模型检验

为了确定模型预测残差是否表现出系统性偏差，采用 t 检验来分析残差均值与 0 是否存在显著差异。在 95%的置信水平下，如果 p 值小于 0.05，这意味着残差的均值统计上不显著地偏离 0，从而表明模型没有显著的系统误差。

6. 2. 10 七到九月销量预测结果展示

对于四种销量数据，我们针对每种数据的特征分别采用了四种模型以提高预测准确性，预测的结果见表 6，完整结果填写在 result 表格中。

表 6:日销量预测结果

	7.1	7.11	7.21	7.31	8.11	8.21	8.31	9.11	9.21
category1	20	25	29	27	29	23	29	23	27
category31	0	0	0	0	0	0	0	0	0
category61	3674	1885	1802	1820	1810	1861	1828	1870	1836
category91	9	2	2	2	2	2	2	1	2
category121	2411	909	908	907	887	921	887	900	895
category151	3803	1899	2094	1839	2140	1753	2066	1835	2053
category181	9	2	2	3	2	2	3	2	2
category211	74	15	13	21	13	17	22	17	15
category241	467	758	762	819	760	790	777	800	740
category271	291	148	159	154	155	152	159	148	157
category301	81	24	21	24	23	22	26	22	24
category331	93	54	49	97	48	55	94	54	52

七、问题二模型的建立与求解

7.1 问题二模型的建立

为了优化仓库管理中的品类分配，本文建立了一个 0-1 整数线性规划模型^[2]，通过科学的规划方案来实现租金成本最小化、产品关联度最大化、仓容利用率最大化以及产能利用率最大化。在本模型的构建过程中，我们假设每个品类只能存放于一个仓库中，同时需要确保仓库容量和产能的限制不被突破。

7.1.1 决策变量

存储规划矩阵 $(x_{ij})_{350 \times 140}$:表示规划方案，其中 x_{ij} 取值为 0 或 1，定义如下:

- $x_{ij}=1$:表示品类 i 存放于仓库 j 中;
- $x_{ij}=0$:表示品类 i 未存放于仓库 j 中;

矩阵表示为:

$$(x_{ij})_{350 \times 140} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1(140)} \\ x_{21} & x_{22} & \cdots & x_{2(140)} \\ \vdots & \vdots & \ddots & \vdots \\ x_{(350)1} & x_{(350)2} & \cdots & x_{(350)(140)} \end{bmatrix}, x_{ij} = 0 \text{ or } 1 \quad (21)$$

7.1.2 目标函数

(1) 主要目标函数

为了满足实际需求，本优化模型构建两个主要目标函数，使得模型能够在满足实际需求的前提下，实现仓库的高效管理与成本控制，确保规划方案的经济性和合理性。

- **最小化租金成本**:仓库的租金成本是影响整个仓储成本的重要因素仓库 j 的日租金为 E_j ，定义函数 $Sig(x)$ 用于表示是否使用某仓库:

$$Sig(x) = \begin{cases} 1, & \text{if } x \geq 1 \\ 0, & \text{otherwise} \end{cases} \quad (22)$$

最小化仓库租金成本的目标函数为:

$$\text{minimize} \sum_{j=1}^{140} Sig\left(\sum_{i=1}^{350} x_{ij}\right) \cdot E_j \cdot 93 \quad (23)$$

该目标函数表示在 93 天内所有被使用仓库的总租金，确保租金成本最低。

- **最大化产品关联度**:通过将关联度高的品类尽可能放在同一个仓库中，可以提高仓储管理效率。假设产品 a 和产品 b 的关联度为 $R(a,b)$ ，定义 $R(a,a) = 0$ ，仓库 j 中的产品集合记为 S_j 。则最大化产品关联度的目标为:

$$\text{maximize} \sum_{j=1}^{140} Sig\left(\sum_{i=1}^{350} x_{ij}\right) \cdot \sum_{a,b \in S_j} R(a,b) \quad (24)$$

(2) 次要目标函数:

在仓储规划中, 合理使用仓库容量和产能是提升仓库效率的重要目标。我们通过构建两个目标函数来最大化仓容利用率和产能利用率。

● 最大化仓容利用率:

$$\text{maximize} \sum_{j=1}^{140} (C_j \sum_{i=1}^{350} x_{ij} \cdot I_{im}) \quad (25)$$

式中, C_j 表示第 j 个仓库的最大存储容量, x_{ij} 表示品类 i 是否存储在仓库 j 中, I_{im} 表示品类 i 在第 m 月的库存量。

通过对所有仓库的容量利用率进行加总, 来最大化整体仓库的空间利用效率, 从而确保仓库资源得到最优使用, 减少浪费。

● 最大化产能利用率:

$$\text{maximize} \sum_{j=1}^{140} (P_j \sum_{i=1}^{350} x_{ij} \cdot O_{in}) \quad (26)$$

式中, P_j 表示第 j 个仓库的最大产能, x_{ij} 表示品类 i 是否存储在仓库 j 中, O_{in} 表示品类 i 在第 n 天的日销量。

通过最大化仓容和产能利用率, 可以提高仓库整体的效率和经济性, 合理利用资源, 避免浪费。

7.1.3 约束条件

● 单一仓库存放约束:

每个品类只能存放在一个仓库中:

$$\sum_{j=1}^{140} x_{ij} = 1, \forall i = 1, 2, \dots, 350 \quad (27)$$

● 仓库容量约束:

每个仓库中所有品类的总库存量不能超过仓库的容量上限 C_j :

$$\sum_{i=1}^{350} x_{ij} \cdot I_{im} \leq C_j, \forall j = 1, 2, \dots, 140, m = 1, 2, 3 \quad (28)$$

● 仓库产能约束:

每个仓库中所有品类的日销量总量不能超过其产能上限 P_j :

$$\sum_{i=1}^{350} x_{ij} \cdot O_{in} \leq P_j, \forall j = 1, 2, \dots, 140, n = 1, 2, \dots, 93 \quad (29)$$

7.1.4 模型总结

结合以上的目标函数和约束条件，0-1 整数线性规划模型可以总结如下

$$\begin{aligned}
 & \min_{1 \leq j \leq 140} \left\{ \text{Sig} \left(\sum_{i=1}^{350} x_{ij} \right) \cdot E_j \cdot 93 \right\} \\
 & \max \left\{ \sum_{j=1}^{140} \left[\text{Sig} \left(\sum_{i=1}^{350} x_{ij} \right) \cdot \sum_{a,b \in S} R(a,b) \right] \right\} \\
 & \max_{1 \leq j \leq 140} \left\{ \sum_{i=1}^{350} x_{ij} \cdot I_{im} / C_j \right\} \\
 & \max_{1 \leq j \leq 140} \left\{ \sum_{i=1}^{350} x_{ij} \cdot O_{in} / P_j \right\} \\
 & s. t. \begin{cases} \sum_{j=1}^{140} x_{ij} = 1 \\ \sum_{i=1}^{350} x_{ij} \cdot I_{im} \leq C_j \\ \sum_{i=1}^{350} x_{ij} \cdot O_{in} \leq P_j \end{cases}
 \end{aligned} \tag{30}$$

7.2 问题二模型的求解

本题的优化目标不唯一，且每个目标的量纲不易去除，因此不适合使用权值将多个目标合并成一个目标函数。为了更好地处理这一问题，我们采用了非支配关系进行解的比较。以两个主要目标为例，比较的方法如下：

如果解 1 的成本 (Expense) 小于解 2 的成本，且解 1 的产品关联度 (Relation) 大于解 2 的关联度，则认为解 1 支配解 2。我们使用符号 $<$ 表示支配关系， $A < B$ 即表示解 A 支配解 B。

7.2.1 多目标粒子群优化算法

为求解该多目标优化问题，我们采用了多目标粒子群优化 (MOPSO) 算法。为了便于求解，使用向量 $x = (x_1, x_2, \dots, x_{350})$ 表示解，其中 $x_i = 1, 2, \dots, 140$ 。具体的求解步骤如下：

Step 1: 初始化种群

首先，对种群进行初始化。初始化的内容包括每个粒子的位置 (Pos_i)、速度 (v_i)、个体极值 ($PBest_i$) 和全局极值 ($GBest_i$)。位置和速度在合理的范围内进行随机赋值，而个体极值和全局极值初始设为粒子的当前位置和初始解。

Step 2: 迭代更新

在达到最大迭代次数之前，对种群中的每个粒子进行迭代更新。具体过程如下：

- 更新粒子速度和位置: 对于每个粒子和每个维度，使用以下公式更新速度和位置：

$$v_{jk} = w \cdot v_{jk} + r_1 \cdot (PBest_{jk} - Pos_{jk}) + r_2 \cdot (GBest_{jk} - Pos_{jk}) \tag{31}$$

其中, w 为惯性权重, r_1 和 r_2 为随机数, 用于增加搜索的随机性, $PBest_{jk}$ 和 $GBest_{jk}$ 分别为个体最优位置和全局最优位置。

位置更新公式为:

$$Pos_{jk} = Pos_{jk} + int(v_{jk} + \epsilon) \quad (32)$$

式中, int 表示取整操作, ϵ 为微小的扰动项, 用于增加位置更新的多样性。

更新完成后, 将位置进行取整, 以确保解在定义域范围内。

● 约束处理

使用约束处理方法来确保粒子的位置满足模型的约束条件。如果某个粒子的位置不满足容量和产能的约束条件, 我们采用最小冲突法进行调整。具体而言, 将超出部分的货物随机分配到其他仓库, 直到满足仓库的容量和产能约束。这种调整方法速度较快, 可以显著减少算法单次迭代的时间。

● 更新解集

- ✧ 对于每个粒子, 计算其当前的目标函数值。
- ✧ 比较当前粒子与解集中的解, 移除解集中被当前粒子支配的解。
- ✧ 将当前粒子加入解集中, 若解集的大小超过设定的限制, 则执行解集截断操作, 以保持解集的合理规模。

Step 3: 更新个体极值和全局极值

- 更新个体极值: 对于每个粒子, 如果当前解优于其个体极值 ($PBest_i$), 则更新该粒子的个体极值为当前解。
- 更新全局极值: 在解集中选择表现最优的解作为全局极值 ($GBest$), 以指导整个种群向更优的方向搜索。采用随机选择的方法, 从解集中选取最优解, 保证种群的多样性。

Step 4: 约束处理方法详述

在每次迭代中, 如果某个粒子的位置不满足模型的约束条件 (如容量和产能限制), 我们参考最小冲突法进行调整。具体方法如下:

- 首先, 将粒子的位置向量转换为一个表示仓库与品类分配的矩阵。
- 对每个仓库, 计算仓库中的货物总容量和总产能, 若超过仓库的上限, 则随机选择超出部分的品类, 将其重新分配到另一个随机选择的仓库中, 直到满足仓库容量和产能的限制条件。
- 这种方法可以确保约束条件得到快速调整, 从而减少迭代过程中由于约束违反带来的复杂性。

Step 5: 解集截断

随着迭代次数的增加, 解集可能会变得过于庞大, 影响计算效率, 因此需要对解集进行截断操作, 以删除密集的解, 保持解集的规模。

● 删除密集解

对解集中每一对解, 计算它们之间相同元素的数量, 若相同元素的数量超过设定的阈值, 则删除其中一个解。这样可以有效减少解集中过于相似的解, 保持解集的多样性。

● 随机删除多余解

如果解集的大小仍然超过设定值, 则随机删除一些解, 直到解集的大小满足要求。这种随机删除方法可以保持解集的随机性和代表性, 避免因集中删除而带来的偏差。

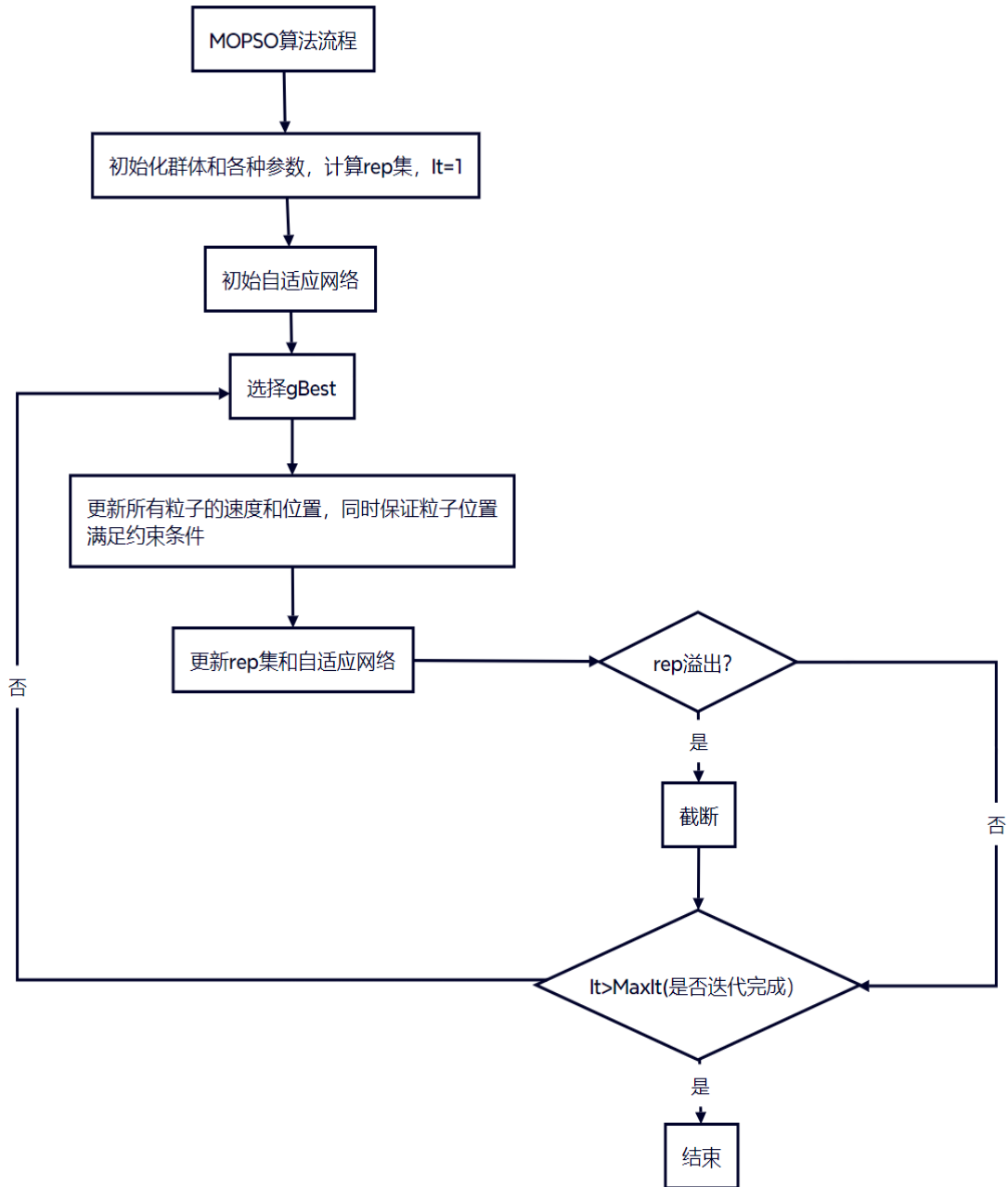


图 13 多目标粒子群优化算法流程

7.2.2 最终解的选择

在所有迭代结束后，选择整个群体中的全局极值作为最终的解，得到最优的品类分仓方案。通过多目标粒子群优化算法（MOPSO），能够有效地优化多个目标，平衡各个目标函数之间的关系，实现仓储网络中成本最小化和效率最大化的优化目标。该方法确保了解的多样性，同时在不同的目标之间实现了有效的权衡，为仓储分配问题提供了切实可行的解决方案。

7.3 结果展示

表 7 “一品一仓”分仓方案

	warehouse
category1	warehouse103
category31	warehouse40
category61	warehouse105
category91	warehouse18
category121	warehouse103
category151	warehouse120
category181	warehouse116
category211	warehouse105
category241	warehouse116
category271	warehouse105
category301	warehouse40
category331	warehouse104

7.4 结果可行性分析

从图 13 中可以分析得出，所使用仓库的仓容利用率和产能利用率均超过 80%，其中接近一半的仓库利用率达到了 90%以上。这表明，模型的分仓方案在提升仓库的资源利用效率方面表现出色，在提高了仓容利用率的同时，又优化了产能利用率。这种高效利用不仅能够降低仓储成本，还能最大限度地提升仓库的整体运营效益。因此，模型在仓库的分配与资源利用优化方面具备较强的实用性和可靠性。

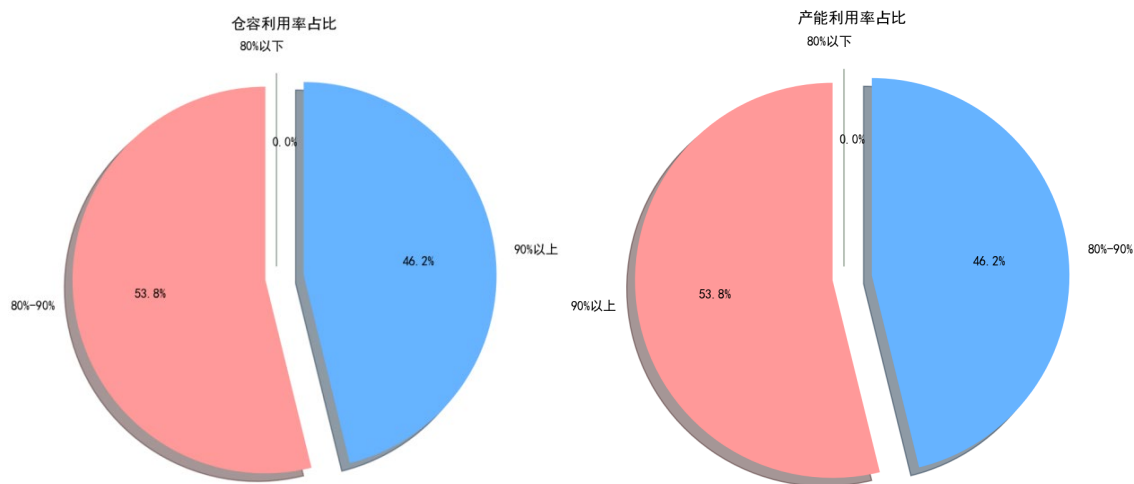


图 14 仓库不同仓容利用率和产能利用率的仓库数量占比

图 14 为论文中的要求给出的分仓方案的示例商品所使用的仓库的仓容利用率、产能利用率和总关联度柱形统计图，完整数据请见附件 3、4。

从图 14 中可以得出，所使用的仓库的产能利用率和仓容利用率均较高，其中大部分仓库的利用率超过了 80%，并且接近一半的仓库达到了 90%以上的利用率。同时，示例商品选择的仓库具有较高的品类关联度，所有仓库的关联度均达到了 2000 以上，总租金成本为 387629.61 元。这表明，模型在规划分仓方案时，不仅实现了较高的品类关联度和成本控制，还显著提高了仓库的产能和仓容利用率。

整体而言，该模型求解结果在实现成本最小化、仓库关联度最大化以及仓库利用率优化方面均有不错表现，展现了良好的多目标优化能力。

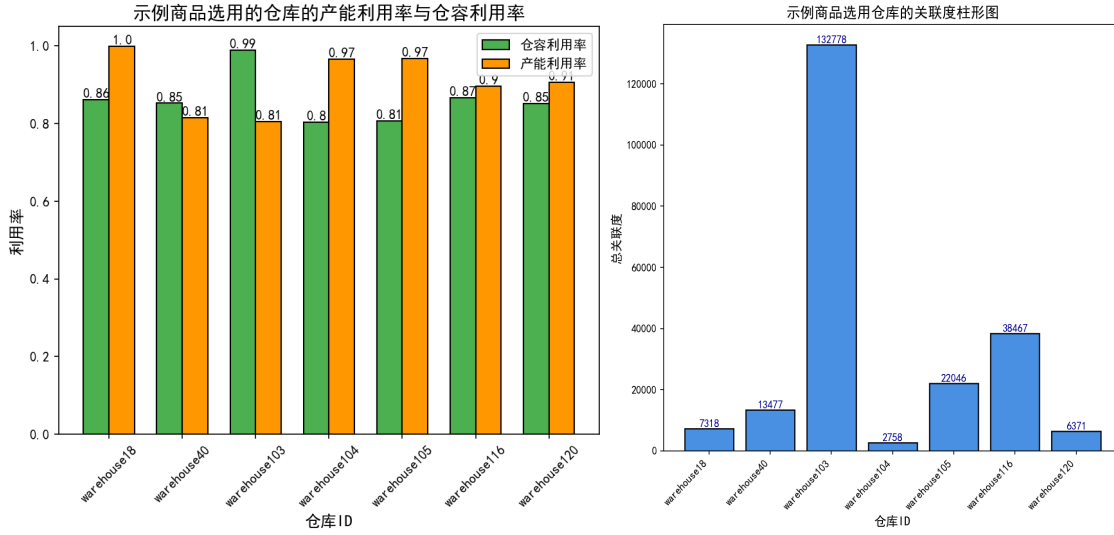


图 15 仓库的仓容利用率、产能利用率和总关联度柱形统计图

八、问题三模型的建立与求解

8.1 问题三模型的建立

问题三中，我们考虑了品类可以在多个仓库中分布存放的情形，并建立了一个线性规划模型。该模型通过优化品类在仓库中的存放比例，旨在最大化仓库资源的利用效率、最小化运营成本，并强化品类间的关联性，以提升整体供应链的响应速度和降低物流成本。

8.1.1 决策变量

决策变量 x_{ij} 代表品类 i 在仓库 j 中的存放比例，构建一个 350×140 的决策矩阵。每个元素 x_{ij} 的取值范围为 $[0,1]$ ，它们表明品类相对于仓库容量的分配比例。矩阵的表达为：

$$(x_{ij})_{350 \times 140} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1(140)} \\ x_{21} & x_{22} & \cdots & x_{2(140)} \\ \vdots & \vdots & \ddots & \vdots \\ x_{(350)1} & x_{(350)2} & \cdots & x_{(350)(140)} \end{pmatrix} \quad (33)$$

8.1.2 目标函数

(1) 最小化租金成本

仓库租金通常是仓库操作的主要固定成本之一，因此通过计算所有被使用仓库的租金总和来降低在给定期限内使用的仓库的总租金成本。计算公式为：

$$\text{minimize} \sum_{j=1}^{140} \text{Sig} \left(\sum_{i=1}^{350} x_{ij} \right) \cdot E_j \cdot 93 \quad (34)$$

式中 E_j 表示仓库 j 的日租金， $\text{Sig}(x)$ 是一个指示函数，用于表示仓库是否被使用。

(2) 最大化产品关联度

通过优化品类在仓库中的存放配置，最大化品类间的关联度，减少处理时间和成本。

$$\text{maximize} \sum_{j=1}^{140} \text{Sig} \left(\sum_{i=1}^{350} x_{ij} \right) \cdot \sum_{a,b \in S_j} R(a,b) \quad (35)$$

式中 $R(a,b)$ 表示品类 a 和品类 b 之间的关联度， S_j 是在仓库 j 中存放的品类集合。

(3) 最大化仓容和产能利用率

目标函数确保每个仓库的存储容量和产能被最大化利用，减少资源浪费，提高经济效益。

$$\begin{cases} \text{maximize} \sum_{j=1}^{140} \left(\sum_{i=1}^{350} x_{ij} \cdot I_{im} \right) \\ \text{maximize} \sum_{j=1}^{140} \left(\sum_{i=1}^{350} x_{ij} \cdot O_{in} \right) \end{cases} \quad (36)$$

式中 I_{im} 表示品类 i 在第 m 月的库存需求，而 O_{in} 表示第 n 日的出库需求。

(4) 最小化分仓数量

为了简化管理和操作，减少品类分布到的仓库数量是另一个关键目标。通过减少仓库的数量，可以降低管理复杂性，简化物流和供应链操作。计算公式如下：

$$\text{minimize} \max_{1 \leq i \leq 350} \left\{ \sum_{j=1}^{140} [x_{ij}] \right\} \quad (37)$$

8.1.3 约束条件

(1) 品类分配完整性约束：

每个品类的存放比例之和必须等于 1，确保每个品类都被完全分配到一个或多个仓库中。

$$\sum_{j=1}^{140} x_{ij} = 1, \forall i = 1, 2, \dots, 350 \quad (38)$$

(2) 存放数量限制

每个品类最多可以存放在三个仓库内。这一约束使用向上取整函数来确保即使是极小的存放比例也被视为该仓库存放了该品类。

$$\sum_{j=1}^{140} [x_{ij}] \leq 3, \forall i = 1, 2, \dots, 350 \quad (39)$$

(3) 仓库容量约束

一个仓库内存放的所有品类的库存量不应该超过该仓库的容量上限。这确保仓库不会过载，从而避免可能的运营问题。

$$\sum_{i=1}^{350} x_{ij} \cdot I_{im} \leq C_j, \forall j = 1, 2, \dots, 140; m = 1, 2, 3 \quad (40)$$

其中， I_{im} 表示品类*i*在第*m*个月的库存需求， C_j 是仓库*j*的容量上限。

(4) 仓库产能约束

仓库中所有品类的出库量不应超过其产能上限。这帮助确保仓库能够应对出库需求，不会因为超载而导致运营效率下降。

$$\sum_{i=1}^{350} x_{ij} \cdot O_{in} \leq P_j, \forall j = 1, 2, \dots, 140; n = 1, 2, \dots, 93 \quad (41)$$

式中， O_{in} 表示品类*i*在第*n*天的出库需求， P_j 是仓库*j*的产能上限。

8.1.4 模型总结

在问题三中，针对品类可分布于多个仓库且需要平衡库存量与出库量比例的需求，我们采用线性规划模型进行优化，结合以上目标函数和约束条件，模型总结如下：

$$\begin{aligned} & \max \left\{ \sum_{j=1}^{140} \left[\text{Sig} \left(\sum_{i=1}^{350} x_{ij} \right) \cdot \sum_{a,b \in S} R(a,b) \right] \right\} \\ & \min_{1 \leq j \leq 140} \left\{ \text{Sig} \left(\sum_{i=1}^{350} x_{ij} \right) \cdot E_j \cdot 93 \right\} \\ & \max_{1 \leq j \leq 140} \left\{ \sum_{i=1}^{350} x_{ij} \cdot I_{im} \right\} \\ & \max_{1 \leq j \leq 140} \left\{ \sum_{i=1}^{350} x_{ij} \cdot O_{in} \right\} \\ & \min_{1 \leq i \leq 350} \left\{ \sum_{j=1}^{140} [x_{ij}] \right\} \\ & s. t. \begin{cases} \sum_{j=1}^{140} x_{ij} = 1 \\ \sum_{i=1}^{350} x_{ij} \cdot I_{im} \leq C_j \\ \sum_{i=1}^{350} x_{ij} \cdot O_{in} \leq P_j \end{cases} \end{aligned} \quad (42)$$

8.2 问题三模型求解

Step 1: 粒子位置更新与分配比例标准化

假设我们有一个粒子表示为六元组 $(w_1, x_1, w_2, x_2, w_3, x_3)$ ，其中 w_i 表示仓库编号， x_i 表示该仓库的分配比例($i = 1, 2, 3$)。我们首先需要对粒子的位置进行更新，并确保分配比例的总和为 1。

- 粒子的位置更新

在多目标粒子群优化 (MOPSO) 中，粒子的更新公式如下：

$$\begin{aligned} v_i(t+1) &= \omega v_i(t) + c_1 r_1(p_i - x_i(t)) + c_2 r_2(g_i - x_i(t)) \\ x_i(t+1) &= x_i(t) + v_i(t+1) \end{aligned} \quad (43)$$

式中， $v_i(t)$ 第 t 次迭代中粒子的速度， $x_i(t)$ 第 t 次迭代中粒子的位置。

在这一过程中，粒子的速度会影响其位置更新，确保粒子能够在解空间中探索。

- 分配比例标准化

在位置更新完成后，为保证分配比例的和为 1，需要对分配比例进行标准化。标准化过程如下：

计算更新后各仓库比例之和：

$$S = x_1 + x_2 + x_3 \quad (44)$$

如果 $S = 0$ ，则保持原分配比例不变；否则，进行标准化处理：

$$x_1' = \frac{x_1}{S}, x_2' = \frac{x_2}{S}, x_3' = \frac{x_3}{S} \quad (45)$$

通过标准化处理，确保三个仓库的分配比例和始终等于 1，从而符合实际分配的需求。

Step 2: 群体极值选取策略

问题三的首要目标是最大化品类关联度。因此，在选取群体极值时，我们优先考虑品类关联度较大的解。

- 评估粒子关联度

首先，需要评估每个粒子的品类关联度，假设品类关联度的评估函数为 $R(v_i)$ ，表示粒子 v 的品类关联度。对所有粒子进行关联度的评估：

$$E_i = R(v_i), \quad \forall v_i \in Solutions \quad (46)$$

- 关联度排序并选择群体极值

将所有解按照品类关联度从大到小进行排序，得到排序后的解集：

$$Solutions_{sorted} = Sort(Solutions, by RelationDegree) \quad (47)$$

然后，在前 K 个解中随机选取一个解作为群体极值：

$$g = Solutions_{sorted}[i], i \in Random(1, K) \quad (48)$$

● 同伴型不同仓库的惩罚机制

对于题目的要求:同一高级品类和件型应尽量放在同一仓库内, 本文构建建立了一个惩罚机制对总关联度减少一定的值作为惩罚, 具体公式如下:

$$Sum_R \leftarrow Sum_R - \omega_1 \cdot \delta_1 - \omega_2 \cdot \delta_2 \quad (49)$$

其中, Sum_R 为总关联度值, δ_1 为每个高级品类所用的仓库数的和, δ_2 为每个件型所用的仓库数的和, ω_1 和 ω_2 分别为比例参数。当同高级品类和同伴型的品类过于分散时, 通过降低其总关联度这个目标函数值将这个解进行淘汰。

Step3:最终解的确定

在所有粒子都更新完位置并选择了新的群体极值后, 选择当前群体极值作为模型的最终解:

$$Best\ Solution = g \quad (50)$$

优先考虑品类关联度较大的粒子, 并结合 MOPSO 的更新策略, 对问题三模型进行求解, 实现对仓储分配的高效管理与决策。

8.3 问题三结果展示

表 8 “一品多仓”分仓方案

	warehouse	warehouse	warehouse
category1	warehouse31	warehouse46	warehouse47
category31	warehouse17		
category61	warehouse39	warehouse36	
category91	warehouse8		
category121	warehouse113		
category151	warehouse19	warehouse124	warehouse12
category181	warehouse121		
category211	warehouse31	warehouse39	warehouse79
category241	warehouse4	warehouse47	warehouse3
category271	warehouse27	warehouse62	warehouse123
category301	warehouse44	warehouse47	warehouse115
category331	warehouse62	warehouse64	

注:表中未填项表示该品类只用一个或两个仓库

8.4 问题三结果可行性分析

从图中可以看出, 不同高级品类和件型的仓库分布比例呈现出良好的优化效果, 具体分析如下:

(1) 高级品类的合理分配:每个仓库中高级品类的数量至五种以下。这种策略显著提升了仓库内部品类的关联性, 有助于优化库存管理和提高操作效率。

(2) 件型的集中管理:模型的求解结果通过将件型数量控制在两种以内, 简化了仓储的管理流程, 减少了管理上的复杂性和潜在的错误率。

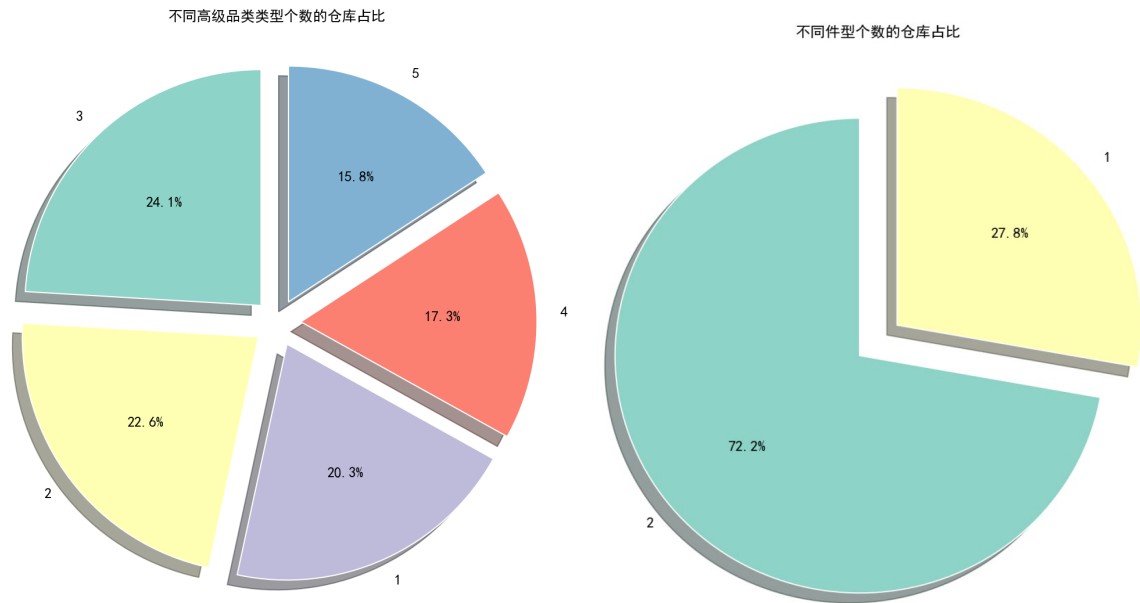


图 16 不同高级品类和件型个数的仓库占比

(3) 仓库总关联度的提升:在尽量将同高级品类和同伴型商品放入同一仓库的策略下,图显示了部分仓库的总关联度情况。尽管总关联度普遍较高,部分仓库的关联度达到了 20000 以上,最高为 35720,表明策略有效地提升了仓库内商品的整体关联性。

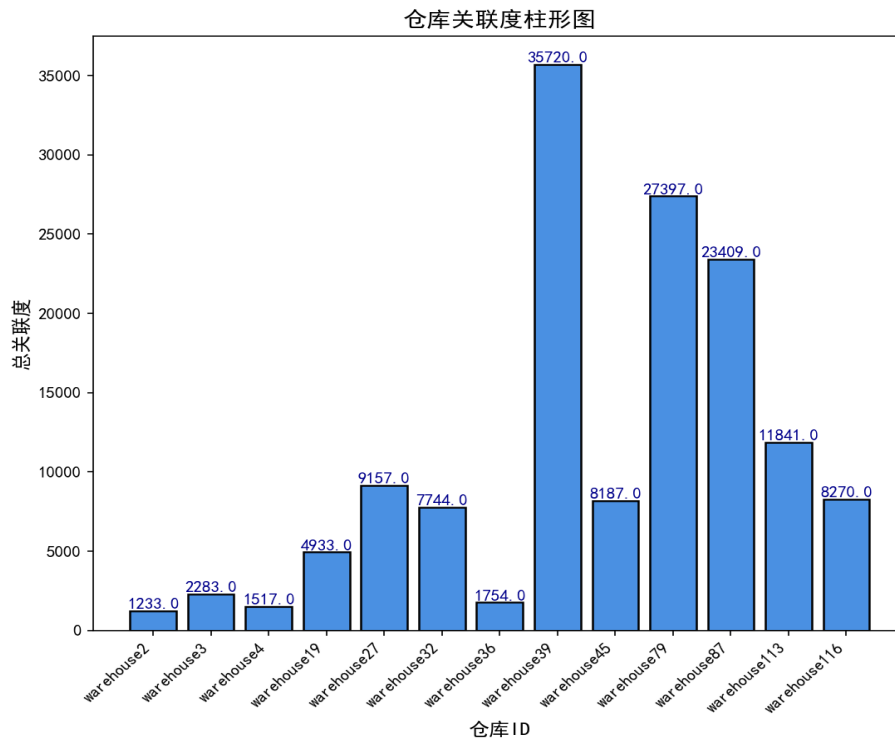


图 17 部分仓库的总关联度柱形图

综上所述,通过对不同高级品类和件型的优化分配,模型的分配结果不仅提升了仓库的管理效率,也确保了高效的库存运用和快速的货物流通,证明了模型的分配结果具有高度可行性。

九、模型的评价、改进与推广

9.1 模型的优点

- 1.对销量数据进行分类并使用适合的模型进行预测，预测的准确度高
- 2.使用启发式算法对复杂且解空间庞大的线性规划问题求解，搜索效率更高
- 3.多目标优化算法给出的解集包含多种情况下的最优解，可根据实际需求选取，应用较为广泛

9.2 模型的缺点

- 1.问题一数据量较小，机器学习算法容易出现过拟合现象。
- 2.问题二、三的求解算法不能收敛到最优解

9.3 模型的推广

该模型具有良好的扩展性，通过对约束条件和目标函数的适当修改，可以应用于其他涉及仓库布局优化的问题，如运输成本、派送成本等。具体来说，该模型可以广泛应用于供应链管理领域，例如大型零售商、分销商或制造商的库存预测和管理。在电子商务企业中，模型能够优化物流网络的布局，提升配送效率和准确性，从而增强用户体验和竞争力。

此外，对于制造业而言，该模型可以用于生产调度和原材料库存管理，帮助企业更合理地规划生产流程，降低成本，提高整体运营效率。因此，该模型不仅适用于当前的问题场景，还可以推广到更广泛的供应链和物流优化中，通过灵活调整模型结构和参数，实现不同业务场景下的最优解决方案。

十、参考文献

- [1] Cai R, Xie S, Wang B, et al. Wind speed forecasting based on extreme gradient boosting[J]. IEEE Access, 2020, 8: 175063-175069.
- [2] Goh M, Kwan J, Zhan Y, et al. A survey of linear programming: Techniques and applications. Operations Research Perspectives, 2021, 8: 100125.
- [3]周志华. (2016). 机器学习. 清华大学出版社, 北京
- [4]武炜杰. 随机森林的应用与优化方法研究[D]. 江南大学, 2021
- [5]刘星, 黄辉. 基于 Prophet 模型的时间序列预测方法研究. 统计与决策, 2021, 37(3): 67-70.

附录

附录 1

介绍：支撑材料的文件列表

附件 1 程序代码文件
附件 2 按销量特征分类并插值后的数据文件
附件 3 问题二各仓库产能利用率、仓容利用率文件
附件 4 问题二各仓库总关联度文件
附件 5 问题三各仓库不同高级品类和件型数量文件
附件 6 问题三各仓库总关联度文件
附件 7 问题三各仓库存放的商品文件
附件 8 问题三各仓库仓容利用率、产能利用率文件

附录 2

介绍：python 代码 数据预处理，进行线性插值

```
import pandas as pd
import glob
import os
file_paths = glob.glob('input/*.csv')
for file_path in file_paths:
    df = pd.read_csv(file_path, encoding='gbk')
    df['日期'] = pd.to_datetime(df['日期'])
    # 生成完整的日期范围
    start_date = df['日期'].min()
    end_date = df['日期'].max()
    all_dates = pd.DataFrame({'日期': pd.date_range(start=start_date, end=end_date)})

    df = pd.merge(all_dates, df, on='日期', how='left')

    df['销量'] = df['销量'].interpolate()

    output_path = os.path.join('input/', os.path.basename(file_path))
    df.to_csv(output_path, index=False, encoding='gbk')
```

附录 3

介绍：python 代码 库存预测模型建立

```
import pandas as pd
import os
import numpy as np
import chardet
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression
```

```

from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
# 自动检测文件编码并读取数据
def read_file(file_path):
    with open(file_path, 'rb') as f:
        result = chardet.detect(f.read())
        encoding = result['encoding']
    df = pd.read_csv(file_path, encoding=encoding, header=None)
    return df

# 存储预测结果
results = []
models = {
    '线性回归': LinearRegression(),
    '随机森林回归': RandomForestRegressor(n_estimators=100, random_state=42),
    '支持向量回归': SVR(),
    'ARIMA': ARIMA,
    'SARIMA': SARIMAX
}

# 遍历所有文件
for i in range(1, 351):
    file_path = f'C:/pythonProject/妈妈杯大数据建模/category {i}.csv'

    if not os.path.exists(file_path):
        print(f'文件 {file_path} 不存在，跳过。')
        continue

    df = read_file(file_path)

    # 将第一列转换为日期，第二列为库存
    df[0] = pd.to_datetime(df[0], format='%Y/%m/%d', errors='coerce')
    df.set_index(0, inplace=True)
    monthly_data = df.resample('ME').sum() # 按月份重采样

    # 生成特征和目标变量
    monthly_data['month'] = monthly_data.index.month
    monthly_data['year'] = monthly_data.index.year
    monthly_data['quarter'] = (monthly_data['month'] - 1) // 3 + 1 # 季度特征
    monthly_data['month_sin'] = np.sin(2 * np.pi * monthly_data['month'] / 12)
    monthly_data['month_cos'] = np.cos(2 * np.pi * monthly_data['month'] / 12)

```

```

# 将目标变量转换为数值类型
y = pd.to_numeric(monthly_data[1], errors='coerce')

# 检查目标变量方差是否为 0
if np.var(y) == 0:
    print(f'目标变量方差为 0，跳过文件 {file_path}...')
    continue

# 检查并去掉任何 NaN 值
monthly_data = monthly_data[~y.isnull()]
y = y.dropna().values # 确保 y 是 numpy 数组

# 生成特征
X = monthly_data[['month', 'quarter', 'month_sin', 'month_cos']]

# 检查特征方差并剔除方差为 0 的特征
X = X.loc[:, X.var() != 0]

# 如果没有可用特征，跳过
if X.shape[1] == 0:
    print(f'所有特征方差全为 0，跳过文件 {file_path}...')
    continue

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 使用完整数据进行训练和验证
best_model_name = None
best_mse = float('inf')
best_r2 = float('-inf')

# 评估每个模型
for model_name, model in models.items():
    if model_name == 'ARIMA':
        try:
            # 使用 ARIMA 模型进行训练和预测
            model_fit = model(y, order=(1, 1, 1)).fit()
            y_pred = model_fit.predict(start=0, end=len(y) - 1)
        except Exception as e:
            print(f'{model_name} 模型在文件 {file_path} 中出错: {e}')
            continue
    elif model_name == 'SARIMA':
        try:
            # 使用 SARIMA 模型进行训练和预测

```

```

        model_fit = model(y, order=(1, 1, 1), seasonal_order=(1, 1, 1,
12)).fit()

        y_pred = model_fit.predict(start=0, end=len(y) - 1)
    except Exception as e:
        print(f'{model_name} 模型在文件 {file_path} 中出错: {e}')
        continue
    else:
        # 使用其他模型
        model.fit(X_scaled, y) # 使用完整数据训练模型
        y_pred = model.predict(X_scaled) # 使用相同数据进行验证

    # 计算均方误差和 R2 分数
    mse = mean_squared_error(y, y_pred) # 计算均方误差
    r2 = r2_score(y, y_pred)

    print(f'{model_name} 在文件 {file_path} 的均方误差: {mse}, R2 分数:
{r2}')

    # 检查预测的方差是否为 0
    if np.var(y_pred) == 0:
        print(f'{model_name} 的预测方差为 0, 排除该模型...')
        continue

    # 记录最佳模型
    if mse < best_mse:
        best_mse = mse
        best_r2 = r2
        best_model_name = model_name

    # 如果没有最佳模型, 跳过预测
    if best_model_name is None:
        print(f'没有找到有效的最佳模型, 跳过文件 {file_path}...')
        continue

    # 进行未来几个月的预测
    if best_model_name in ['ARIMA', 'SARIMA']:
        # 使用最佳 ARIMA 或 SARIMA 模型预测未来 3 个月
        future_predictions = model_fit.forecast(steps=3)
    else:
        future_months = pd.DataFrame({
            'month': [7, 8, 9], # 预测 7 月到 9 月
            'quarter': [3, 3, 3], # 假设季度为 3
            'month_sin': np.sin(2 * np.pi * np.array([7, 8, 9]) / 12),
            'month_cos': np.cos(2 * np.pi * np.array([7, 8, 9]) / 12)

```

```

    })
    future_months_scaled = scaler.transform(future_months) # 对未来数据进行
缩放
    future_predictions = models[best_model_name].predict(future_months_scaled)

# 将预测结果添加到结果列表
result_row = {
    '品类': i,
    '最佳模型': best_model_name,
    '七月库存量': int(future_predictions[0]),
    '八月库存量': int(future_predictions[1]),
    '九月库存量': int(future_predictions[2])
}

results.append(result_row)

# 将结果转换为 DataFrame 并排序
results_df = pd.DataFrame(results)
results_df.sort_values(by='品类', inplace=True)

# 保存结果
results_df.to_csv('predicted_inventory_best_model.csv', index=False, encoding='utf-8-
sig')
print("预测结果已保存至 predicted_inventory_best_model.csv")

```

附录 4

介绍：python 代码 preohet 模型预测 4-6 与 7-8 销量差距过大类

```

import pandas as pd
import matplotlib.pyplot as plt
from prophet import Prophet
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.model_selection import ParameterGrid
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
data = pd.read_csv('category3.csv', encoding='ANSI')
data['日期'] = pd.to_datetime(data['日期']) # 确保日期列转换为 datetime 类型
data.set_index('日期', inplace=True)

# 特征工程：构造新的时间特征
data['年'] = data.index.year
data['月'] = data.index.month
data['日'] = data.index.day
data['周'] = data.index.isocalendar().week

```

```

data['季度'] = data.index.quarter
data['星期'] = data.index.dayofweek # 星期一为 0, 星期日为 6

# 添加后 92 个数据特征
data['后 92 特征'] = 0
data.loc[data.index[-92:], '后 92 特征'] = 1 # 后 92 个数据的特征

# 生成加权销量特征
data['加权销量'] = data['销量']
data.loc[data.index[-92:], '加权销量'] *= 1.5 # 给后 92 个数据增加权重, 1.5 倍

# 删除包含 NaN 的行
data.dropna(inplace=True)

# 定义节假日和 618 促销日期
holidays = pd.DataFrame({
    'holiday': '节假日',
    'ds': pd.to_datetime(
        ['2022-10-01', '2022-10-02', '2022-10-03', '2022-10-04', '2022-10-05', '2022-10-06', '2022-10-07',
         '2023-05-01', '2023-05-02', '2023-05-03',
         '2023-06-17', '2023-06-18', '2023-06-19']), # 618 及其前后
    'lower_window': 0,
    'upper_window': 1,
})

# 使用加权销量进行建模
prophet_data = data.reset_index().rename(columns={'日期': 'ds', '加权销量': 'y'})

# 定义参数网格
param_grid = {
    'seasonality_prior_scale': [10.0, 100.0, 500, 1000],
    'changepoint_prior_scale': [10.0, 100.0, 500, 1000],
    'holidays_prior_scale': [10.0, 100.0, 300],
    'n_changepoints': [100, 200, 500],
    'seasonality_mode': ['multiplicative'],
}

# 存储最佳参数和最低误差
best_params = None
lowest_mse = float('inf')

# 网格搜索
for params in ParameterGrid(param_grid):

```

```

# 初始化 Prophet 模型
prophet_model = Prophet(
    holidays=holidays,
    **params
)

# 拟合模型
prophet_model.fit(prophet_data)

# 使用原训练集的日子进行预测
forecast = prophet_model.predict(prophet_data[['ds']])

# 计算误差指标
mse = mean_squared_error(prophet_data['y'], forecast['yhat'])

# 更新最佳参数
if mse < lowest_mse:
    lowest_mse = mse
    best_params = params

# 输出最佳参数
print("最佳参数:")
print(best_params)
print(f'最低均方误差 (MSE): {lowest_mse}')

# 使用最佳参数拟合最终模型
final_model = Prophet(holidays=holidays, **best_params)
final_model.fit(prophet_data)
final_forecast = final_model.predict(prophet_data[['ds']])

# 预测未来的销量（2023 年 7 月到 9 月）
future_dates = pd.date_range(start='2023-07-01', end='2023-09-30')
future_df = pd.DataFrame({'ds': future_dates})

# 使用最终模型进行未来预测
future_forecast = final_model.predict(future_df)

# 可视化结果
plt.figure(figsize=(12, 6))
plt.plot(data.index, data['销量'], label='实际销量', color='black')
plt.plot(data.index, final_forecast['yhat'], label='最佳模型预测', color='blue')
plt.plot(future_forecast['ds'], future_forecast['yhat'], label='未来预测', color='orange') #
未来预测
plt.legend()

```

```

plt.title('Prophet 模型预测与实际销量对比')
plt.xlabel('日期')
plt.ylabel('销量')
plt.show()

import seaborn as sns
from scipy.stats import skew, kurtosis

# 计算残差
residuals = prophet_data['y'] - final_forecast['yhat']

# 绘制残差图
plt.figure(figsize=(12, 6))
plt.scatter(prophet_data['ds'], residuals, color='blue', alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.title('残差图', fontproperties='SimHei')
plt.xlabel('日期', fontproperties='SimHei')
plt.ylabel('残差', fontproperties='SimHei')
plt.show()

# 计算统计特征
mean_residual = residuals.mean()
std_residual = residuals.std()
skewness = skew(residuals)
kurt = kurtosis(residuals)

# 输出统计特征
print(f'残差均值: {mean_residual:.2f}')
print(f'残差标准差: {std_residual:.2f}')
print(f'残差偏度: {skewness:.2f}')
print(f'残差峰度: {kurt:.2f}')

```

附录 5

介绍：python 代码 XGBoost 模型预测日趋势过大类销量

```

import pandas as pd
import numpy as np
import xgboost as xgb
from sklearn.metrics import mean_squared_error, mean_absolute_error
import glob
# 读取所有以"category"开头的 CSV 文件
file_list = glob.glob('category*.csv')

# 创建一个空的 DataFrame 用于存储所有品类的预测结果
all_predictions = pd.DataFrame()

```

```

for file in file_list:
    # 从 CSV 文件中读取数据，指定编码为 ANSI
    data = pd.read_csv(file, encoding='ANSI')
    data['日期'] = pd.to_datetime(data['日期']) # 确保日期列转换为 datetime 类型
    data.set_index('日期', inplace=True)
    # 创建时间特征
    data['month'] = data.index.month
    data['day'] = data.index.day
    data['dayofweek'] = data.index.dayofweek
    data['year'] = data.index.year
    # 创建滞后特征
    data['销量_lag1'] = data['销量'].shift(1) # 前一天的销量
    data['销量_lag7'] = data['销量'].shift(7) # 前七天的销量
    # 创建滚动特征
    data['销量_rolling_mean_3'] = data['销量'].rolling(window=3).mean() # 过去三
天的平均销量
    data['销量_rolling_mean_7'] = data['销量'].rolling(window=7).mean() # 过去七
天的平均销量

    # 删除缺失值
    data.dropna(inplace=True)

    # 目标变量
    y = data['销量'].values

    # 创建特征矩阵
    X = data[['month', 'day', 'dayofweek', 'year', '销量_lag1', '销量_lag7',
              '销量_rolling_mean_3', '销量_rolling_mean_7']]

    # 创建 XGBoost 数据集
    train_data = xgb.DMatrix(X, label=y)

    # 设置模型参数
    params = {
        'objective': 'reg:squarederror',
        'eval_metric': 'rmse',
        'max_depth': 6,
        'learning_rate': 0.05,
        'subsample': 0.8,
        'colsample_bytree': 0.9,
        'silent': 1
    }

    # 训练模型

```

```

num_round = 2000 # 迭代次数
model = xgb.train(params, train_data, num_round)

# 预测未来的销量（2023 年 7 月到 9 月）
future_dates = pd.date_range(start='2023-07-01', end='2023-09-30')
future_data = pd.DataFrame({
    'month': future_dates.month,
    'day': future_dates.day,
    'dayofweek': future_dates.dayofweek,
    'year': future_dates.year
})

# 使用训练数据的最后一条记录来填充未来数据的滞后特征
last_row = data.iloc[-1]
future_data['销量_lag1'] = last_row['销量']
future_data['销量_lag7'] = last_row['销量']

# 处理未来数据的滚动特征
future_data['销量_rolling_mean_3'] = future_data['销量_lag1'].rolling(window=3).mean()
future_data['销量_rolling_mean_7'] = future_data['销量_lag1'].rolling(window=7).mean()

# 创建未来数据的 XGBoost 数据集
future_data_dmatrix = xgb.DMatrix(future_data)

# 使用模型进行预测
future_predictions = model.predict(future_data_dmatrix)
future_predictions = np.round(future_predictions).astype(int)

# 将预测结果保存到 DataFrame
all_predictions[file] = future_predictions

# 设置列名为日期
all_predictions.columns = [file.split('/')[-1].replace('.csv', '') for file in file_list]
all_predictions.index = pd.date_range(start='2023-07-01', end='2023-09-30')

# 保存所有预测结果到一个 CSV 文件
all_predictions.to_csv('all_category_predictions.csv', index_label='日期')

```

附录 6

介绍：python 代码 分位数回归模型预测销量数据过少类

```

import pandas as pd
import numpy as np

```

```

import glob
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib.pyplot as plt
from statsmodels.regression.quantile_regression import QuantReg
from scipy import stats
import matplotlib.font_manager as fm
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
# 初始化一个空的 DataFrame 来存储所有品类的预测结果
all_predictions = pd.DataFrame()

# 获取所有以"category"开头的 CSV 文件
file_list = glob.glob('category*.csv')

for file in file_list:
    data = pd.read_csv(file, encoding='ANSI')
    data['日期'] = pd.to_datetime(data['日期'])
    data.set_index('日期', inplace=True)

    y = data['销量'].values
    if np.all(y == 0):
        print(f'销量全为零，使用默认值进行预测: {file}")
        future_dates = pd.date_range(start='2023-07-01', end='2023-09-30')
        future_predictions = np.zeros(len(future_dates), dtype=int) # else:
        # 创建设计矩阵
        X = np.arange(len(data)).reshape(-1, 1) # 使用索引作为特征
        X = sm.add_constant(X) # 添加常数项

        # 使用分位数回归进行建模（例如 50%的分位数）
        quantile_model = QuantReg(y, X)
        result = quantile_model.fit(q=0.5) # 50%分位数

        # 预测未来销量
        future_X = np.arange(len(data), len(data) + 92).reshape(-1, 1) # 92 天
        future_X = sm.add_constant(future_X) # 添加常数项

        future_predictions = result.predict(future_X)

        # 确保预测结果为非负整数
        future_predictions = np.clip(np.round(future_predictions), 0, None).astype(int)

    # 计算残差
    residuals = y - result.predict(X)

```

```

# 进行 t 检验
t_stat, p_value = stats.ttest_1samp(residuals, 0)
print(f'{file} 的 t 检验结果: t 统计量 = {t_stat}, p 值 = {p_value}')
# 绘制残差图
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.scatter(data.index, residuals, color='blue', alpha=0.5)
plt.axhline(0, color='red', linestyle='--')
plt.title(f'{file} 的残差图')
plt.xlabel('日期')
plt.ylabel('残差')

# 绘制回归曲线
plt.subplot(1, 2, 2)
plt.scatter(data.index, y, color='blue', label='实际销量', alpha=0.5)
plt.plot(data.index, result.predict(X), color='orange', label='分位数回归预测',
linewidth=2)
plt.title(f'{file} 的分位数回归曲线')
plt.xlabel('日期')
plt.ylabel('销量')
plt.legend()

plt.tight_layout()
plt.savefig(f'residuals_and_regression_{file.split("/")[-1].replace(".csv",
"" )}.png')
plt.show()
all_predictions[file.split('/')[-1].replace('.csv', '')] = future_predictions
all_predictions.to_csv('sales_forecasts.csv', index=False)

```

附录 7

介绍: python 代码 prophet 模型预测可时序分析类

```

# 导入必要的库
import pandas as pd
import matplotlib.pyplot as plt
from prophet import Prophet
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import ParameterGrid
from statsmodels.tsa.seasonal import seasonal_decompose
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 从 CSV 文件中读取数据, 指定编码为 ANSI
data = pd.read_csv('category1.csv', encoding='ANSI')
data['日期'] = pd.to_datetime(data['日期']) # 确保日期列转换为 datetime 类型

```

```

data.set_index('日期', inplace=True)

# 特征工程：构造新的时间特征
data['年'] = data.index.year
data['月'] = data.index.month
data['日'] = data.index.day
data['周'] = data.index.isocalendar().week
data['季度'] = data.index.quarter
data['星期'] = data.index.dayofweek # 星期一为 0，星期日为 6

# 滞后特征
data['销量滞后 1'] = data['销量'].shift(1) # 前一天销量
data['销量滞后 2'] = data['销量'].shift(2) # 前两天销量

# 滚动特征
data['销量滚动均值 7'] = data['销量'].rolling(window=7).mean() # 过去 7 天的滚动均值
data['销量滚动均值 30'] = data['销量'].rolling(window=30).mean() # 过去 30 天的滚动均值

data.dropna(inplace=True)

# 定义节假日和 618 促销日期
holidays = pd.DataFrame({
    'holiday': '节假日',
    'ds': pd.to_datetime(
        ['2022-10-01', '2022-10-02', '2022-10-03', '2022-10-04', '2022-10-05', '2022-10-06', '2022-10-07',
         '2023-05-01', '2023-05-02', '2023-05-03',
         '2023-06-17', '2023-06-18', '2023-06-19']), # 618 及其前后
    'lower_window': 0,
    'upper_window': 1,
})

# 使用全部数据进行建模
prophet_data = data.reset_index().rename(columns={'日期': 'ds', '销量': 'y'})

# 定义参数网格
param_grid = {
    'seasonality_prior_scale': [10.0, 100.0, 500, 1000],
    'changepoint_prior_scale': [10.0, 100.0, 500, 1000],
    'holidays_prior_scale': [10.0, 100.0, 300],
    'n_changepoints': [100, 200, 500],
    'seasonality_mode': ['multiplicative'],

```

```

}

# 存储最佳参数和最低误差
best_params = None
lowest_mse = float('inf')

# 网格搜索
for params in ParameterGrid(param_grid):
    prophet_model = Prophet(holidays=holidays, **params)
    prophet_model.fit(prophet_data)
    forecast = prophet_model.predict(prophet_data[['ds']])
    mse = mean_squared_error(prophet_data['y'], forecast['yhat'])

    if mse < lowest_mse:
        lowest_mse = mse
        best_params = params

# 输出最佳参数
print("最佳参数:")
print(best_params)
print(f'最低均方误差 (MSE): {lowest_mse}')

# 使用最佳参数拟合最终模型
final_model = Prophet(holidays=holidays, **best_params)
final_model.fit(prophet_data)
final_forecast = final_model.predict(prophet_data[['ds']])

# 预测未来的销量（2023 年 7 月到 9 月）
future_dates = pd.date_range(start='2023-07-01', end='2023-09-30')
future_df = pd.DataFrame({'ds': future_dates})
future_forecast = final_model.predict(future_df)

# 可视化结果
plt.figure(figsize=(12, 6))
plt.plot(data.index, data['销量'], label='实际销量', color='black')
plt.plot(data.index, final_forecast['yhat'], label='最佳模型预测', color='blue')
plt.plot(future_forecast['ds'], future_forecast['yhat'], label='未来预测', color='orange')
plt.fill_between(future_forecast['ds'],
                 final_forecast['yhat_lower'],
                 final_forecast['yhat_upper'],
                 color='lightblue', alpha=0.5, label='95% 置信区间')

plt.legend()
plt.title('预测区间')
plt.xlabel('日期')

```

```

plt.ylabel('销量')
plt.show()

# 绘制残差图
residuals = prophet_data['y'] - final_forecast['yhat']
plt.figure(figsize=(12, 6))
plt.scatter(final_forecast['ds'], residuals, color='red')
plt.axhline(y=0, color='black', linestyle='--')
plt.title('残差图')
plt.xlabel('日期')
plt.ylabel('残差')
plt.show()

# 绘制趋势图
plt.figure(figsize=(12, 6))
plt.plot(final_forecast['ds'], final_forecast['trend'], label='趋势成分', color='blue')
plt.title('Prophet 模型的趋势成分')
plt.xlabel('日期')
plt.ylabel('趋势')
plt.legend()
plt.show()

## 提取季节性成分并绘制
# seasonal = final_forecast[['ds', 'seasonal']]
# plt.figure(figsize=(12, 6))
# plt.plot(seasonal['ds'], seasonal['seasonal'], label='季节性成分', color='green')
# plt.title('Prophet 模型的季节性成分')
# plt.xlabel('日期')
# plt.ylabel('季节性成分')
# plt.legend()
# plt.show()

# 绘制周期性图
data['月'] = data.index.month
monthly_sales = data.groupby('月')['销量'].sum()
plt.figure(figsize=(12, 6))
plt.plot(monthly_sales.index, monthly_sales.values, marker='o', linestyle='-')
plt.title('每月销量总和')
plt.xlabel('月份')
plt.ylabel('销量')
plt.xticks(monthly_sales.index)
plt.show()

# 时间序列分解

```

```

decomposition = seasonal_decompose(data['销量'], model='additive', period=30)
trend = decomposition.trend
seasonal = decomposition.seasonal
residual = decomposition.resid

plt.figure(figsize=(12, 12))
plt.subplot(3, 1, 1)
plt.plot(trend, label='趋势', color='blue')
plt.legend()

plt.subplot(3, 1, 2)
plt.plot(seasonal, label='季节性', color='green')
plt.legend()

plt.subplot(3, 1, 3)
plt.plot(residual, label='残差', color='red')
plt.legend()

plt.tight_layout()
plt.show()

# 绘制移动平均图
data['销量滚动均值 30'] = data['销量'].rolling(window=30).mean()
plt.figure(figsize=(12, 6))
plt.plot(data.index, data['销量'], label='实际销量', color='black')
plt.plot(data.index, data['销量滚动均值 30'], label='30 天滚动均值', color='orange')
plt.legend()
plt.title('销量与 30 天滚动均值对比')
plt.xlabel('日期')
plt.ylabel('销量')
plt.show()

```

附录 8

介绍：python 代码 问题二求解代码

```

import pandas as pd
import pulp
import numpy as np
# 读取数据
warehouses = pd.read_csv('附件 3.csv', encoding='gbk')
category_info = pd.read_csv('附件 5.csv', encoding='gbk')
category_assoc = pd.read_csv('附件 4.csv', encoding='gbk')

# 创建品类到索引的映射表，将品类编码映射到整数索引
category_list = pd.concat([category_assoc['品类 1'], category_assoc['品类 2']]).unique()

```

```

if len(category_list) > 350:
    raise ValueError("品类数量超过 350，请检查数据。")

# 映射表
category_to_index = {category: idx for idx, category in enumerate(category_list)}
index_to_category = {v: k for k, v in category_to_index.items()}

# 初始化关联度矩阵为 350*350
assoc_matrix = np.zeros((350, 350))
print(assoc_matrix.shape)

# 填充关联度矩阵
for _, row in category_assoc.iterrows():
    category1 = row['品类 1']
    category2 = row['品类 2']

    if category1 in category_to_index and category2 in category_to_index:
        idx1 = category_to_index[category1]
        idx2 = category_to_index[category2]
        assoc_matrix[idx1, idx2] = row['关联度']
        assoc_matrix[idx2, idx1] = row['关联度'] # 矩阵是对称的

# 输出填充后的关联度矩阵
print("关联度矩阵填充完成。")

# 读取新的库存预测文件
inventory_predictions = pd.read_excel('库存预测结果.xlsx', engine='openpyxl')
inventory_predictions.columns = ['品类', '七月库存量', '八月库存量', '九月库存量']
inventory_predictions = inventory_predictions.melt(id_vars=['品类'], var_name='月份',
value_name='预测库存量')

def extract_month(month_str):
    if '七月' in month_str:
        return 7
    elif '八月' in month_str:
        return 8
    elif '九月' in month_str:
        return 9
    else:
        return None
inventory_predictions['月份'] = inventory_predictions['月份'].apply(extract_month)

```

```

max_inventory = inventory_predictions.groupby('品类')['预测库存量']
max_inventory.reset_index()
max_inventory.columns = ['品类', '最大库存量']

sales_predictions = pd.read_excel('商品销量预测.xlsx', header=None,
engine='openpyxl')
num_days = sales_predictions.shape[1] - 1
sales_predictions.columns = ['商品名'] + [f'7 月 {i} 日' for i in range(1, num_days + 1)]
sales_predictions = sales_predictions.melt(id_vars=['商品名'], var_name='日期',
value_name='预测销量')

max_sales = sales_predictions.groupby('商品名')['预测销量'].max().reset_index()
max_sales.columns = ['品类', '预测销量']

# 确保分类信息的列名正确
if len(category_info.columns) == 3:
    category_info.columns = ['品类', '高级品类', '件型']
elif len(category_info.columns) == 2:
    category_info.columns = ['品类', '高级品类']

# 确保仓库数据的列名正确
warehouses.columns = ['仓库 ID', '仓容上限', '每日最大出库量', '租金成本']
# 提取品类编号
max_inventory['品类'] = max_inventory['品类'].str.extract(r'(\d+)', expand=False)
max_sales['品类'] = max_sales['品类'].str.extract(r'(\d+)', expand=False)
# warehouses['商品 ID'] = warehouses['商品 ID'].str.extract(r'(\d+)', expand=False)
# 提取数字并转换为整数
# warehouses['仓库 ID'] = warehouses['仓库 ID'].str.extract(r'(\d+)',
expand=False).astype(int)

# print(assoc_matrix)
# 初始化优化问题
model = pulp.LpProblem("Warehouse_Optimization", pulp.LpMaximize)

# 定义决策变量
x = pulp.LpVariable.dicts("CategoryInWarehouse",
[(i, j) for i in max_inventory['品类'] for j in
warehouses['仓库 ID']],
cat='Binary')

# 定义指示变量，用于标识仓库是否被利用
y = pulp.LpVariable.dicts("WarehouseUtilized", warehouses['仓库 ID'], cat='Binary')
category_to_index = {category: idx for idx, category in enumerate(max_inventory['品类'])}

```

```

# 目标函数
for j in warehouses['仓库 ID']:
    inventory_sum = pulp.lpSum([
        x[i, j] * max_inventory.loc[max_inventory['品类'] == i, '最大库存量']
    ].values[0]
    for i in max_inventory['品类']
    if not max_inventory.loc[max_inventory['品类'] == i, '最大库存量'].empty
    ])

    sales_sum = pulp.lpSum([
        x[i, j] * max_sales.loc[max_sales['品类'] == i, '预测销量'].values[0]
        for i in max_inventory['品类']
        if not max_sales.loc[max_sales['品类'] == i, '预测销量'].empty
    ])

    rent_sum = pulp.lpSum([
        warehouses.loc[warehouses['仓库 ID'] == j, '租金成本'].values[0] * x[i, j]
        for i in max_inventory['品类']
    ])

# 计算商品关联度
assoc_sum = pulp.lpSum([
    assoc_matrix[category_to_index[a], category_to_index[b]] * x[a, j] * x[b, j]
    for a in max_inventory['品类']
    for b in max_inventory['品类']
    if a != b and a in category_to_index and b in category_to_index
    and x[a, j].varValue is not None and x[b, j].varValue is not None # 添加赋值
    检查
])

model += (
    inventory_sum / warehouses.loc[warehouses['仓库 ID'] == j, '仓库上限']
    ].values[0] +
    sales_sum / warehouses.loc[warehouses['仓库 ID'] == j, '每日最大出库量']
    ].values[0] - rent_sum + assoc_sum
    ), f'Warehouse_Utilization_{j}"

model.sense = pulp.LpMaximize

# 约束条件
# 1. 每个品类只能存放在一个仓库中
for i in max_inventory['品类']:
    model += pulp.lpSum([x[i, j] for j in warehouses['仓库 ID']]) == 1,
    f'OneWarehouseForCategory_{i}"

```

```

# 2. 仓库的容量限制
for j in warehouses['仓库 ID']:
    model += pulp.lpSum([x[i, j] * max_inventory.loc[max_inventory['品类'] == i, '最大库存量'].values[0]
                        for i in max_inventory['品类']
                        if not max_inventory.loc[max_inventory['品类'] == i, '最大库存量'].empty]) <= warehouses.loc[warehouses['仓库 ID'] == j, '仓容上限'].values[0], f'WarehouseCapacity_{j}"

# 3. 仓库的产能限制
for j in warehouses['仓库 ID']:
    model += pulp.lpSum([x[i, j] * max_sales.loc[max_sales['品类'] == i, '预测销量'].values[0]
                        for i in max_inventory['品类']
                        if not max_sales.loc[max_sales['品类'] == i, '预测销量'].empty]) <= warehouses.loc[warehouses['仓库 ID'] == j, '每日最大出库量'].values[0], f'WarehouseCapacityLimit_{j}"

for j in warehouses['仓库 ID']:
    inventory_utilization = pulp.lpSum([
        x[i, j] * max_inventory.loc[max_inventory['品类'] == i, '最大库存量'].values[0]
        for i in max_inventory['品类']
    ]) / warehouses.loc[warehouses['仓库 ID'] == j, '仓容上限'].values[0]

    sales_utilization = pulp.lpSum([
        x[i, j] * max_sales.loc[max_sales['品类'] == i, '预测销量'].values[0]
        for i in max_inventory['品类']
    ]) / warehouses.loc[warehouses['仓库 ID'] == j, '每日最大出库量'].values[0]
    model += pulp.lpSum([x[i, j] for i in max_inventory['品类']]) <= \
        warehouses.loc[warehouses['仓库 ID'] == j, '仓容上限'].values[0] * y[j], f'WarehouseActive_{j}"
    model += inventory_utilization >= 0.8 * y[j], f'InventoryUtilization_{j}"
    model += sales_utilization >= 0.8 * y[j], f'SalesUtilization_{j}"
model.solve()
print(f'Status: {pulp.LpStatus[model.status]}")
# 输出结果
results = []
for i in max_inventory['品类']:
    for j in warehouses['仓库 ID']:
        if pulp.value(x[i, j]) == 1:
            results.append([i, j])

```

```

results_df = pd.DataFrame(results, columns=['品类', '仓库 ID'])
results_df['品类_numeric'] = results_df['品类'].astype(int) # 新增列用于排序
results_df = results_df.sort_values(by='品类_numeric')
results_df['品类'] = 'category' + results_df['品类'].astype(str)
results_df = results_df.drop(columns=['品类_numeric'])
results_df.to_csv('warehouse_allocation_results.csv', index=False)

# 检查是否所有品类都分配了仓库
assigned_categories = set(results_df['品类'])
all_categories = set(category_info['品类'])
if len(assigned_categories) != len(all_categories):
    print("警告：并非所有品类都有分配仓库。")

print("优化完成，分仓方案已保存为 warehouse_allocation_results2.csv")

# 计算仓库利用率和总仓租成本
utilization_results = []
total_rent_cost = 0
assoc_results = [] # 用于保存每个仓库的关联度

for j in warehouses['仓库 ID']:
    print(j)
    total_inventory = 0
    total_sales = 0
    total_assoc = 0 # 用于计算关联度总和

    for i in max_inventory['品类']:
        if pulp.value(x[i, j]) == 1:
            # 检查品类是否在 max_inventory 中
            if not max_inventory.loc[max_inventory['品类'] == i, '最大库存量'].empty:
                total_inventory += max_inventory.loc[max_inventory['品类'] == i, '最大库存量'].values[0]

            # 检查品类是否在 max_sales 中
            if not max_sales.loc[max_sales['品类'] == i, '预测销量'].empty:
                total_sales += max_sales.loc[max_sales['品类'] == i, '预测销量'].values[0]

    # 计算关联度
    for b in max_inventory['品类']:
        if i != b and pulp.value(x[b, j]) == 1: # 确保不是同一品类且已经选择

```

```

        total_assoc += assoc_matrix[category_to_index[i],
category_to_index[b]]

    print(total_inventory)
    print(total_sales)
    warehouse_capacity = warehouses.loc[warehouses['仓库 ID'] == j, '仓库上限
'].values[0]
    print(warehouse_capacity)
    warehouse_capacity_limit = warehouses.loc[warehouses['仓库 ID'] == j, '每日最大
出库量'].values[0]
    print(warehouse_capacity_limit)
    rent_cost = warehouses.loc[warehouses['仓库 ID'] == j, '租金成本'].values[0]
    print(rent_cost)
    print("-----")
    utilization_rate = total_inventory / warehouse_capacity if warehouse_capacity > 0
else 0
    production_utilization_rate = total_sales / warehouse_capacity_limit if
warehouse_capacity_limit > 0 else 0

    utilization_results.append([j, utilization_rate, production_utilization_rate])
    assoc_results.append([j, total_assoc/2]) # 保存关联度结果
    total_rent_cost += rent_cost if total_inventory > 0 else 0 # 只有在使用该仓库时
才计算租金

# 保存利用率结果
utilization_df = pd.DataFrame(utilization_results, columns=['仓库 ID', '仓库利用率', '产
能利用率'])
utilization_df.to_csv('warehouse_utilization_results2.csv', index=False)

# 保存关联度结果
assoc_df = pd.DataFrame(assoc_results, columns=['仓库 ID', '总关联度'])
assoc_df.to_csv('warehouse_assoc_results2.csv', index=False)

# 输出总仓租成本
print(f'总仓租成本: {total_rent_cost}')

```

附录 9

介绍: python 代码 问题三求解代码

```

import pandas as pd
import pulp
import numpy as np
# 读取数据
warehouses = pd.read_csv('附件 3.csv', encoding='gbk')
category_info = pd.read_csv('附件 5.csv', encoding='gbk')

```

```

category_assoc = pd.read_csv('附件 4.csv', encoding='gbk')

# 创建品类到索引的映射表，将品类编码映射到整数索引
category_list = pd.concat([category_assoc['品类 1'], category_assoc['品类 2']]).unique()

if len(category_list) > 350:
    raise ValueError("品类数量超过 350，请检查数据。")

# 映射表
category_to_index = {category: idx for idx, category in enumerate(category_list)}
index_to_category = {v: k for k, v in category_to_index.items()}

assoc_matrix = np.zeros((350, 350))
print(assoc_matrix.shape)

# 填充关联度矩阵
for _, row in category_assoc.iterrows():
    category1 = row['品类 1']
    category2 = row['品类 2']

    if category1 in category_to_index and category2 in category_to_index:
        idx1 = category_to_index[category1]
        idx2 = category_to_index[category2]
        assoc_matrix[idx1, idx2] = row['关联度']
        assoc_matrix[idx2, idx1] = row['关联度'] # 矩阵是对称的

# 输出填充后的关联度矩阵
print("关联度矩阵填充完成。")

# 读取新的库存预测文件
inventory_predictions = pd.read_excel('库存预测结果.xlsx', engine='openpyxl')
inventory_predictions.columns = ['品类', '七月库存量', '八月库存量', '九月库存量']
inventory_predictions = inventory_predictions.melt(id_vars=['品类'], var_name='月份',
value_name='预测库存量')

def extract_month(month_str):
    if '七月' in month_str:
        return 7
    elif '八月' in month_str:
        return 8
    elif '九月' in month_str:
        return 9
    else:
        return None

```

```

inventory_predictions['月份'] = inventory_predictions['月份'].apply(extract_month)

# 获取最大库存量
max_inventory = inventory_predictions.groupby('品类')['预测库存量']
max_inventory = max_inventory.max().reset_index()
max_inventory = pd.merge(max_inventory, category_info, on='品类', how='left')

max_inventory.columns = ['品类', '最大库存量', '高级品类', '件型']

# 获取最大销量
sales_predictions = pd.read_excel('商品销量预测.xlsx', header=None,
engine='openpyxl')
num_days = sales_predictions.shape[1] - 1
sales_predictions.columns = ['商品名'] + [f'7月{i}日' for i in range(1, num_days + 1)]
sales_predictions = sales_predictions.melt(id_vars=['商品名'], var_name='日期',
value_name='预测销量')

max_sales = sales_predictions.groupby('商品名')['预测销量'].max().reset_index()
max_sales.columns = ['品类', '预测销量']

# 确保分类信息的列名正确
if len(category_info.columns) == 3:
    category_info.columns = ['品类', '高级品类', '件型']
elif len(category_info.columns) == 2:
    category_info.columns = ['品类', '高级品类']

# 确保仓库数据的列名正确
warehouses.columns = ['仓库ID', '仓容上限', '每日最大出库量', '租金成本']

# 提取品类编号
max_inventory['品类'] = max_inventory['品类'].str.extract(r'(\d+)', expand=False)
max_sales['品类'] = max_sales['品类'].str.extract(r'(\d+)', expand=False)

# print(assoc_matrix)
# 初始化优化问题
model = pulp.LpProblem("Warehouse_Optimization", pulp.LpMaximize)
# unique_count = max_inventory['高级品类'].nunique()
# print("不同的高级品类个数:", unique_count)
# unique_count2 = max_inventory['件型'].nunique()
# print("不同的件型个数:", unique_count2)
# 定义决策变量
x = pulp.LpVariable.dicts("CategoryInWarehouse",

```

```

                                [(i, j) for i in max_inventory['品类'] for j in
warehouses['仓库 ID']],
                                lowBound=0, # 下界
                                upBound=1, # 上界
                                cat='Continuous') # 改为连续变量

# 创建二进制变量 y[i, j], 表示商品 i 是否在仓库 j 中
y = pulp.LpVariable.dicts("y", [(i, j) for i in max_inventory['品类'] for j in
warehouses['仓库 ID']], cat='Binary')
# 为每个商品和仓库添加同步约束
for i in max_inventory['品类']:
    for j in warehouses['仓库 ID']:
        model += x[i, j] <= 1 * y[i, j], f'Sync1_{i}_{j}'
        model += x[i, j] >= 0, f'Sync2_{i}_{j}'

category_to_index = {category: idx for idx, category in enumerate(max_inventory['品类'
'])}
# 创建二进制变量表示仓库是否被使用
use_warehouse = pulp.LpVariable.dicts("use_warehouse", warehouses['仓库 ID'],
cat='Binary')
# 目标函数
for j in warehouses['仓库 ID']:
    # 计算商品关联度
    assoc_sum = pulp.lpSum([
        assoc_matrix[category_to_index[a], category_to_index[b]] * y[a, j] * y[b, j]
        for a in max_inventory['品类']
        for b in max_inventory['品类']
        if a != b and a in category_to_index and b in category_to_index
        and y[a, j].varValue is not None and y[b, j].varValue is not None # 添加赋值
检查
    ])

    # 添加约束: 如果仓库被使用, 则关联度不得小于 5000
    # model += pulp.lpSum(x[i, j] for i in max_inventory['品类']) <=
use_warehouse[j]*350, f'UseWarehouse_{j}'
    # model += assoc_sum >= 1000 * use_warehouse[j], f'MinAssocSum_{j}'

item_type_vars = [] # 用于跟踪选择的件型
# 统计每种件型的数量
for item_type in max_inventory['件型'].unique():
    items_by_item_type = max_inventory[max_inventory['件型'] == item_type]['
品类']
    if not items_by_item_type.empty:
        # 计算该件型的商品数量

```

```

        item_count = pulp.lpSum(y[i, j] for i in items_by_item_type)
        # 创建一个二进制变量，表示是否有该件型的商品被选中
        var = pulp.LpVariable(f'item_type_var_{item_type}_{j}', cat='Binary')
        item_type_vars.append(var)
        # 将 item_count 与二进制变量关联
        model += item_count <= 350 * var, f'ItemTypeCount_{item_type}_{j}'
        model += item_count >= var, f'ItemTypeCount_2_{item_type}_{j}'
    model += pulp.lpSum(item_type_vars) <= 2, f'MaxItemTypes_{j}'

    high_category_vars = [] # 用于跟踪选择的高级品类
    for high_category in max_inventory['高级品类'].unique():
        items_by_high_category = max_inventory[max_inventory['高级品类'] ==
high_category]['品类']
        if not items_by_high_category.empty:
            high_category_count = pulp.lpSum(y[i, j] for i in
items_by_high_category)
            var = pulp.LpVariable(f'high_category_var_{high_category}_{j}',
cat='Binary')
            high_category_vars.append(var)
            # 关联高级品类计数
            model += high_category_count <= var * 350,
f'HighCategoryCount_LessThan_{high_category}_{j}' # 添加名称
            model += high_category_count >= var,
f'HighCategoryCount_GreaterThan_{high_category}_{j}' # 添加名称
            model += pulp.lpSum(high_category_vars) <= 5, f'MaxHighCategories_{j}'
            model += assoc_sum-pulp.lpSum(item_type_vars)*10-
pulp.lpSum(high_category_vars)*10, f'Warehouse_Utilization_{j}'
            #-pulp.lpSum(item_type_vars)*10-pulp.lpSum(high_category_vars)*10
    model.sense = pulp.LpMaximize

    for i in max_inventory['品类']:
        use_warehouse = {j: pulp.LpVariable(f'use_{i}_{j}', cat='Binary') for j in
warehouses['仓库 ID']}
        for j in warehouses['仓库 ID']:
            model += x[i, j] <= use_warehouse[j] * (1e6), f'WarehouseUsage_{i}_{j}'
            model += x[i, j] >= use_warehouse[j] * 1e-6,
f'WarehouseUsage_Lower_{i}_{j}'

            model += pulp.lpSum(use_warehouse[j] for j in warehouses['仓库 ID']) <= 3,
f'MaxWarehouses_{i}'
            model += pulp.lpSum(use_warehouse[j] for j in warehouses['仓库 ID']) >= 1,
f'MinWarehouses_{i}'

    for i in max_inventory['品类']:

```

```

    model += pulp.lpSum([x[i, j] for j in warehouses['仓库 ID']] == 1,
f'SumToOne_{i}')

for j in warehouses['仓库 ID']:
    model += pulp.lpSum([x[i, j] * max_inventory.loc[max_inventory['品类'] == i, '最大库存量'].values[0]
                        for i in max_inventory['品类']
                        if not max_inventory.loc[max_inventory['品类'] == i, '最大库存量'].empty]) <= warehouses.loc[warehouses['仓库 ID'] == j, '仓库容量'].values[0], f'WarehouseCapacity_{j}')

# 3. 仓库的产能限制
for j in warehouses['仓库 ID']:
    model += pulp.lpSum([x[i, j] * max_sales.loc[max_sales['品类'] == i, '预测销量'].values[0]
                        for i in max_inventory['品类']
                        if not max_sales.loc[max_sales['品类'] == i, '预测销量'].empty]) <= warehouses.loc[warehouses['仓库 ID'] == j, '每日最大出库量'].values[0],
f'WarehouseCapacityLimit_{j}')

# 求解模型
model.solve()
print(f'Status: {pulp.LpStatus[model.status]}')

# 输出结果
results = []
for i in max_inventory['品类']:
    for j in warehouses['仓库 ID']:
        if pulp.value(x[i, j]) > 0:
            print([i, j])
            results.append([i, j])

# 保存结果为 CSV
results_df = pd.DataFrame(results, columns=['品类', '仓库 ID'])

# 将品类列转换为整数进行排序
results_df['品类_numeric'] = results_df['品类'].astype(int) # 新增列用于排序

# 按照数字排序
results_df = results_df.sort_values(by='品类_numeric')

# 添加前缀
results_df['品类'] = 'category' + results_df['品类'].astype(str)

```

```

# 删除临时排序列
results_df = results_df.drop(columns=['品类_numeric'])

# 保存结果
results_df.to_csv('warehouse_allocation_results.csv', index=False)

# 检查是否所有品类都分配了仓库
assigned_categories = set(results_df['品类'])
all_categories = set(category_info['品类'])
if len(assigned_categories) != len(all_categories):
    print("警告：并非所有品类都有分配仓库。")

print("优化完成，分仓方案已保存为 warehouse_allocation_results.csv")

# 计算仓库利用率和总仓租成本
utilization_results = []
total_rent_cost = 0
assoc_results = [] # 用于保存每个仓库的关联度

for j in warehouses['仓库 ID']:
    print(j)
    total_inventory = 0
    total_sales = 0
    total_assoc = 0 # 用于计算关联度总和

    for i in max_inventory['品类']:
        if pulp.value(x[i, j]) > 0:
            # 检查品类是否在 max_inventory 中
            if not max_inventory.loc[max_inventory['品类'] == i, '最大库存量
'].empty:
                total_inventory += max_inventory.loc[max_inventory['品类'] == i, '
最大库存量'].values[0]*pulp.value(x[i, j])

            # 检查品类是否在 max_sales 中
            if not max_sales.loc[max_sales['品类'] == i, '预测销量'].empty:
                total_sales += max_sales.loc[max_sales['品类'] == i, '预测销量
'].values[0]*pulp.value(x[i, j])

            # 计算关联度
            for b in max_inventory['品类']:
                if i != b and pulp.value(x[b, j]) > 0: # 确保不是同一品类且已经
选择
                    total_assoc += assoc_matrix[category_to_index[i],
category_to_index[b]]

```

```

    print(total_inventory)
    print(total_sales)
    warehouse_capacity = warehouses.loc[warehouses['仓库 ID'] == j, '仓库上限
'].values[0]
    print(warehouse_capacity)
    warehouse_capacity_limit = warehouses.loc[warehouses['仓库 ID'] == j, '每日最大
出库量'].values[0]
    print(warehouse_capacity_limit)
    rent_cost = warehouses.loc[warehouses['仓库 ID'] == j, '租金成本'].values[0]
    print(rent_cost)
    print("-----")

    utilization_rate = total_inventory / warehouse_capacity if warehouse_capacity > 0
else 0
    production_utilization_rate = total_sales / warehouse_capacity_limit if
warehouse_capacity_limit > 0 else 0

    utilization_results.append([j, utilization_rate, production_utilization_rate])
    assoc_results.append([j, total_assoc/2]) # 保存关联度结果
    total_rent_cost += rent_cost if total_inventory > 0 else 0 # 只有在使用该仓库时
才计算租金

# 保存利用率结果
utilization_df = pd.DataFrame(utilization_results, columns=['仓库 ID', '仓库利用率', '产
能利用率'])
utilization_df.to_csv('warehouse_utilization_results.csv', index=False)

# 保存关联度结果
assoc_df = pd.DataFrame(assoc_results, columns=['仓库 ID', '总关联度'])
assoc_df.to_csv('warehouse_assoc_results.csv', index=False)
# 输出总仓租成本
print(f'总仓租成本: {total_rent_cost}')

```